UNIT 4: Section 1:

ADO.Net:

What is ADO.NET?

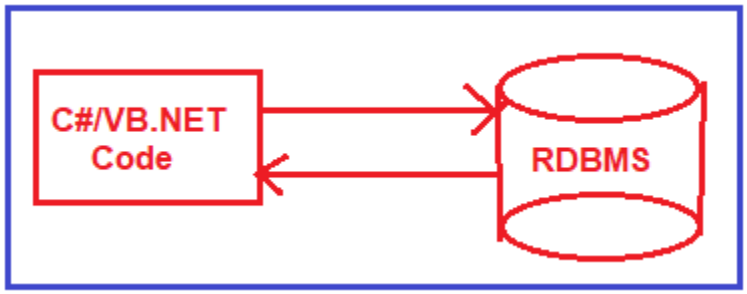
ADO stands for Microsoft ActiveX Data Objects. ADO.NET is one of Microsoft's data access technologies, which we can use to communicate with different data sources. It is a part of or component of the .NET Framework, which establishes a connection between the .NET Application (Console, WCF, WPF, Windows, MVC, Web Form, etc.) and different data sources. The Data Sources can be SQL Server, Oracle, MySQL, XML, etc. ADO.NET consists of a set of predefined classes that can be used to connect, retrieve, insert, update, and delete data (i.e., performing CRUD operation) from data sources. ADO.NET mainly uses **System.Data.dll** and **System.Xml.dll.**

Difference between ADO and ADO.NET:

ADO	ADO.NET
Itis a COM based library.	Itis a CLR based library.
ClassicADO require active connection With the data store.	ADO.NET architecture works while the data store is disconnected.
Locking feature is available.	Locking feature is not available.
Datais stored in Binary format.	Data is stored in XML format.
XML integrationis not possible.	XML integrationis possible.
Ituses the object named Recordset to reference data from the data store.	It uses the Dataset objectfor data access and representation.
Firewall might preventexecution of classic ADO.	ADO.NET has firewall proof and its execution will never be interrupted.
Classic ADOarchitecture includes client side cursor and server side cursor.	ADO.NET architecturedoesn't include such cursor.

ADO.NET (ActiveX Data Objects .NET) is a data access technology in the Microsoft .NET framework that provides a set of libraries and classes for working with data from various sources, including databases, XML files, and more. It is part of the .NET framework's base class library and interacts with data-centric applications and databases.

ADO.NET is designed to provide a bridge between your application code and the underlying data sources. It offers a way to perform tasks like connecting to databases, executing queries, retrieving and updating data, and managing connections and transactions. This is especially important for building data-driven applications and services.



A connection between the .NET Application and different data sources

Namespaces used in ADO.Net

System.Data:
It contains the common classes for connecting, fetching data from database. Classes are like as Data Table , Dataset , Dataview etc.

System.Data.SqlClient:
It contain classes for connecting fetching data from Sql server database. Classes are like as SqlDataAdapter , SqlDataReader etc.

System.Data.OracleClient:
It contain classes for connecting fetching data from Oracle database. Classes are like as OracleDataAdapter , OracleDataReader etc.

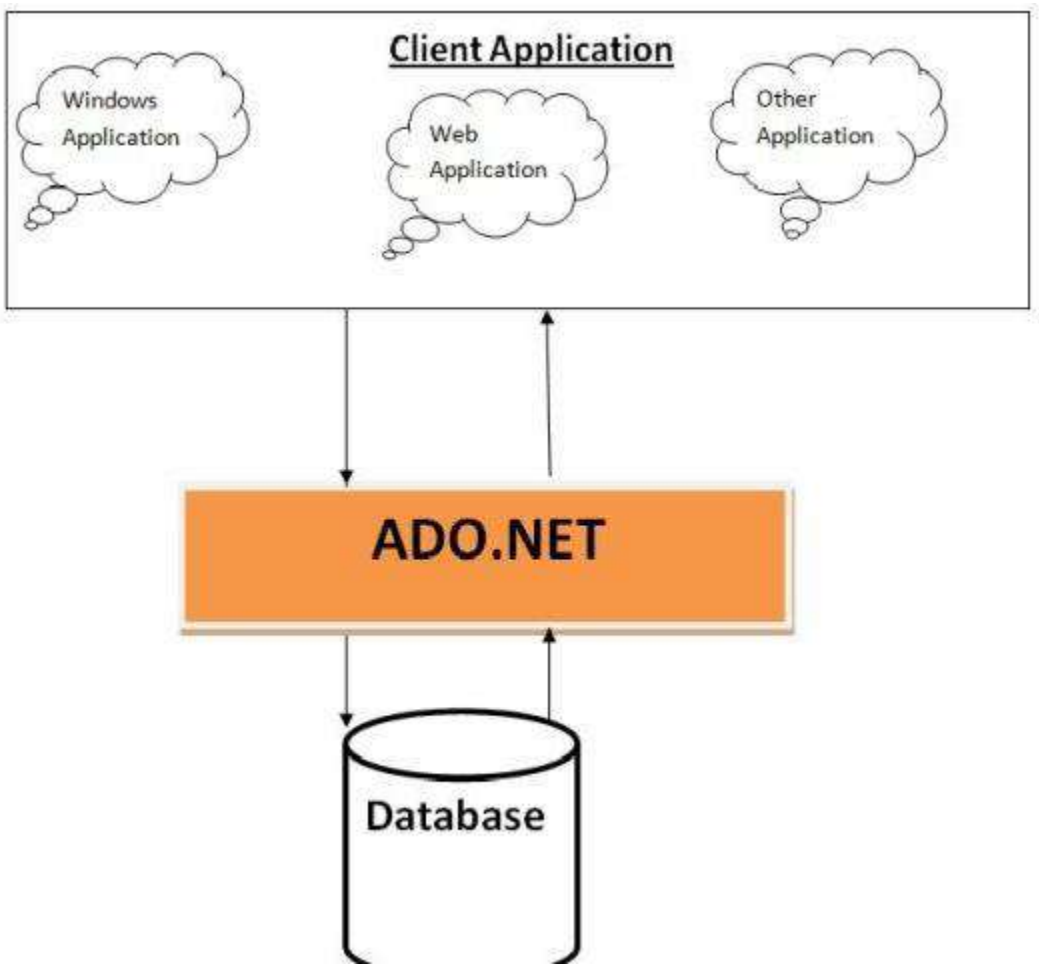
System.Data.OleDb:
It contain classes for connecting , fetching data from any database (like Ms Access , db2 , Oracle , Sqlserver , mySql). Classes are like as OleDbDataAdapter , OleDbDataReader , etc.

What Types of Applications Use ADO.NET?

ADO.NET is used in various applications where data access and manipulation are crucial. Here are some types of applications that commonly use ADO.NET:

- **Desktop or Windows Applications:** Traditional desktop applications like Windows Forms and WPF applications often need to interact with databases or other data sources. ADO.NET provides the necessary tools to connect to databases, retrieve data, and update records.

- **Web Applications:** Web applications, including ASP.NET Web Forms and ASP.NET MVC applications, require data access to display, collect, and manage information. ADO.NET enables these applications to connect to databases and present data to users.
- **Console Applications:** Console applications might need to perform data-related tasks, like importing/exporting data, data analysis, or reporting. ADO.NET can facilitate these tasks by providing efficient data access.
- **Service Applications:** Background or Windows services that process data often rely on ADO.NET to connect to databases and handle data-related operations.



The Architecture of ADO.NET (Components of ADO.NET):

Components are designed for data manipulation and faster data access. **Connection, Command, DataReader, DataAdapter, DataSet,** and **DataView** are the components of ADO.NET that are used to perform database operations.

ADO.NET comprises several key components that work together to facilitate data access and manipulation in .NET applications. These components provide the building blocks for connecting to data sources, executing queries, retrieving and updating data, and managing transactions. Here are the main components of ADO.NET:

- **Connection:** The Connection component establishes a connection to a data source, such as a database. It manages the underlying connection to the database server and provides methods to open and close the connection.

- **Command:** The Command component represents a command that is executed against a data source. It encapsulates SQL statements, stored procedure calls, and other database commands. The two main types of command objects are SQLCommand, which is used for executing SQL queries and stored procedures against SQL Server databases, and OleDbCommand, which is Used for executing commands against OLE DB data sources, which include various database types.

- **DataReader:** The DataReader component efficiently reads data from a data source. It provides a forward-only, read-only stream of data that is particularly useful for retrieving large datasets. Reading data with a DataReader is fast and memory-efficient.

- **DataAdapter:** The DataAdapter bridges the application's DataSet (in-memory cache of data) and the data source. It facilitates the retrieval of data from the data source into the DataSet and also allows changes to be updated in the DataSet back to the data source. Specific DataAdapter classes exist for different data sources, such as SqlDataAdapter and OleDbDataAdapter.

- **DataSet:** The DataSet is an in-memory data cache that can hold multiple tables, relationships, and constraints. It allows disconnected data manipulation, meaning that data is retrieved from the data source, disconnected from the connection, and then manipulated without direct interaction with the data source. The Data Set can be considered an in-memory representation of the database.

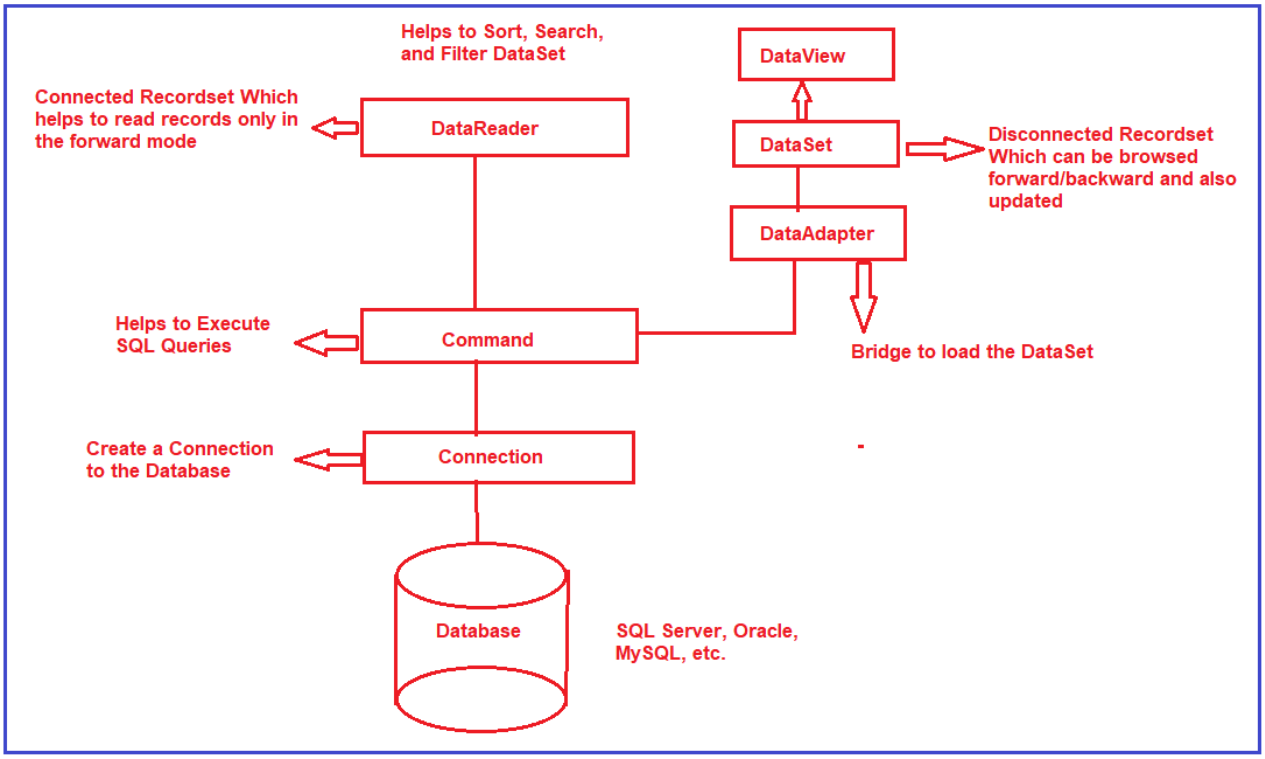
- **DataTable:** A data table is a component within a Data Set that represents a table of data. It consists of rows and columns and allows you to store and manipulate tabular data. DataTables can have relationships and constraints to maintain data integrity.

- **DataView:** The DataView is used to filter, sort, and navigate through data within a DataTable. It provides a dynamic view of the data, allowing you to customize how it is presented to the user.

- **Transaction:** The Transaction component provides support for managing transactions in ADO.NET. Transactions group multiple data access operations into a single unit of work that can be either committed (made permanent) or rolled back (undone) as a whole.

- **Connection String:** The connection string is a configuration string that provides the necessary information to connect to a data source. It includes details such as the database server's location, credentials, and other settings.

From the above components, two components are compulsory. One is the command object and the other one is the connection object. Irrespective of the operations like Insert, Update, Delete and Select, the command and connection object you always need. For a better understanding, please have a look at the following image.

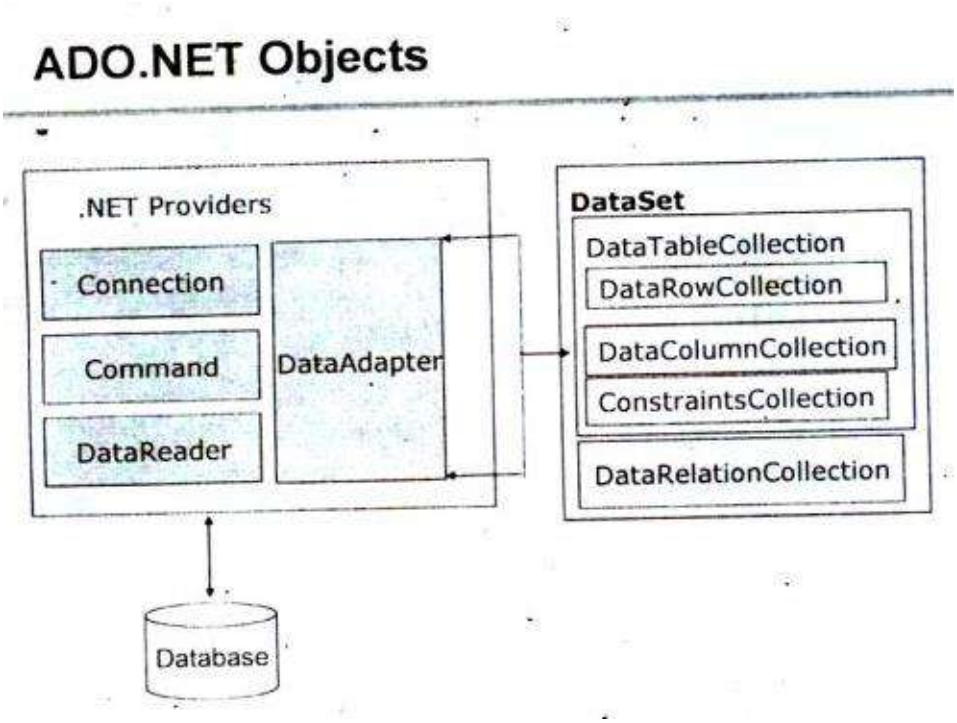


These components work together to enable developers to connect to various data sources, retrieve and manipulate data, and manage transactions efficiently within .NET applications. The

choice of which component to use and how to use them depends on the specific requirements and design of the application.

ADO.NET Object Model

In the ADO.NET object model, the data residing in a database is retrieved through a data provider. The data provider is the set of components including the Connection, Command, DataReader, and DataAdapter objects. An application can access data either through a dataset or through a datareader object.



The ADO.NET object model has two main components that are used for accessing and manipulating data. They are as follows:

1. **Data Provider and**
2. **DataSet.**

The four key components of a data provider are.

1. **Connection:** Used to connect to the data source.
2. **Command:** Used to execute a command against the data source. This component retrieves, inserts, deletes, and modifies data in a data source.
3. **DataReader:** This component retrieves data from a data source in read-only and forward mode.

4. **DataAdapter:** Used to populate a dataset with the data retrieved from a database and to update the data source.

What are ADO.NET Data Providers?

The Database cannot directly execute our C# code; it only understands SQL. So, if a .NET application needs to retrieve data or to do some insert, update, and delete operations from or to a database, then the .NET application needs to

1. **Connect to the Database**
2. **Prepare an SQL Command**
3. **Execute the Command**
4. **Retrieve the results and display them in the application**

This is possible with the help of .NET Data Providers.

ADO.NET Code to Connect with SQL Server Database

The following image shows the sample ADO.NET code, which connects to the SQL Server Database and retrieves data. If you notice in the below image, here, we are using some predefined classes such as **SQLConnection**, **SQLCommand,** and **SQLDataReader.** These classes are called .NET Provider classes, which are responsible for interacting with the database and performing the CRUD operation. If you further notice all the classes are prefixed with the word SQL, it means these classes will interact with only the SQL Server database.

```
SqlConnection connection = new SqlConnection("data source=.; database=TestDB; integrated security=SSPI");
SqlCommand command = new SqlCommand("Select * from Customers", connection);
connection.Open();
SqlDataReader myReader = command.ExecuteReader();

while (myReader.Read())
{
    Console.WriteLine("\t{0}\t{1}", myReader.GetInt32(0), myReader.GetString(1));
}

connection.Close();
```

All these classes are present in the System.Data.SqlClient namespace. We can also say that the .NET data provider for the SQL Server database is the System.Data.SqlClient.

ADO.NET Code to Connect with SQL Database

The following code is for connecting to the Oracle Database and retrieving data. If you notice, we are using SqlConnection, SqlCommand, and SqlDataReader classes here. That means all these classes have prefixed the word Oracle, and these classes are used to communicate with the Oracle database only.

```
SqlConnection connection = new SqlConnection("data source=.; database=TestDB; integrated security=SSPI");
SqlCommand command = new SqlCommand("Select * from Customers", connection);
connection.Open();
SqlDataReader myReader = command.ExecuteReader();
while (myReader.Read())
{
Console.WriteLine("\t{0}\t{1}", myReader.GetInt32(0), myReader.GetString(1));
}
connection.Close();
```

All the above classes are present in the System.Data. SqlClient namespace so that we can say that System.Data. SqlClient is the .NET Data Provider for Oracle Database.

Note: Similarly, if you want to communicate with OLEDB data sources such as Excel, Access, etc., you must use OleDbConnection, OleDbCommand, and OleDbDataReader classes. So, the .NET data provider for OLEDB data sources is the System.Data.OleDb.

Different .NET Data Providers

SQL Server Data Provider: System.Data.SqlClient
Oracle Data Provider: System.Data.OracleClient
OLEDB Data Provider: System.Data.OleDb
ODBC Data Provider: System.Data.Odbc

ADO.NET Data Providers

Please have a look at the following image to better understand the ADO.NET Data Providers. The first section is the .NET Applications, the second section is the .NET Data Providers, and the third section is the data sources. You need to use the appropriate .NET Provider in your application based on the data source.

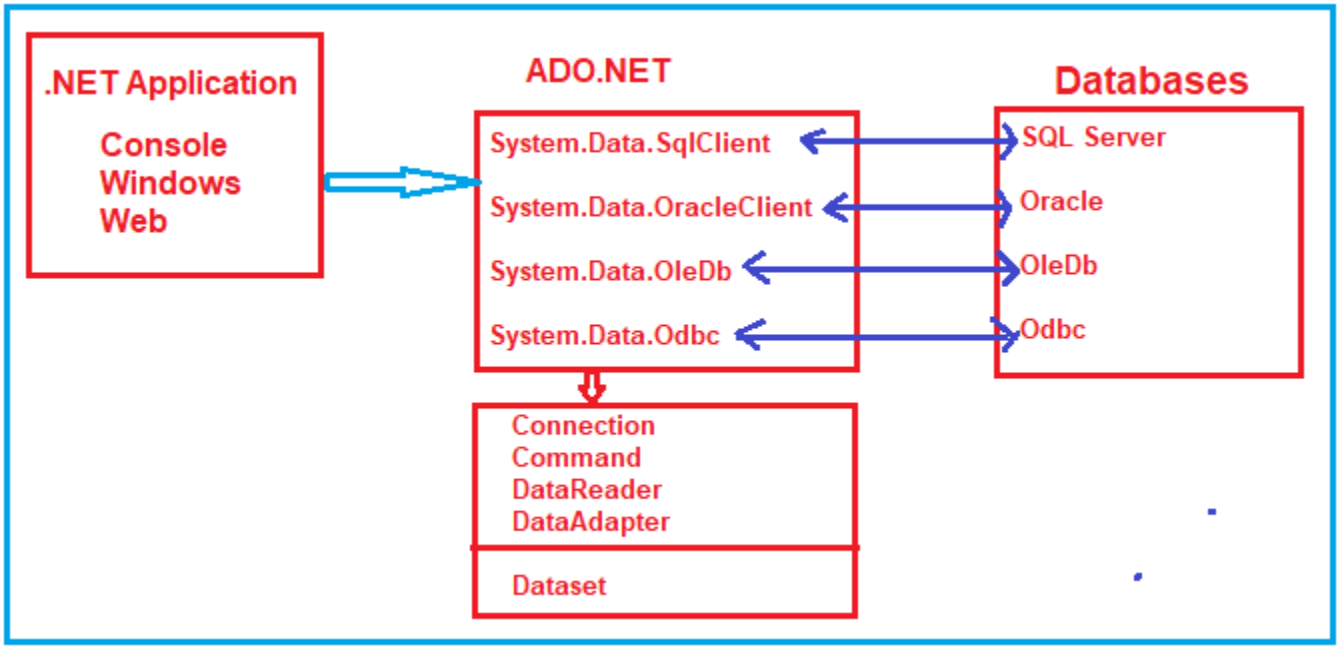
Selecting an appropriate data provider for a client application depends on the type of data source being accessed. There are four .Net data providers available.

SQL Server: It's used to work specifically with Microsoft SQL Server. It exists in a namespace within the System.Data.SqlClient.

OLE DB: It's used to work with the OLEDB provider. The System.Data.dll assembly implements the OLEDB .NET framework data provider in the System.Data.OleDb namespace.

ODBC: To use this type of provider, you must use an ODBC driver. The System.Data.ODBC.dll assembly implements the ODBC .NET framework data provider. This assembly is not part of the Visual Studio .NET installation.

Oracle: The System.Data.OracleClient.dll assembly implements the Oracle .NET framework data provider in the System.Data.OracleClient namespace. The Oracle client software must be installed on the system before you can use the provider to connect to an Oracle data source.



The Connection, Command, DataAdapter, and DataReader objects are provider-specific, whereas the DataSet is provider-independent. That means if you are going to work with the SQL Server database, then you need to use SQL-specific provider objects such as SQLConnection, SqlCommand, SqlDataAdapter, and SqlDataReader objects which belong to the System.Data.SqlClient namespace.

Note: If you understand how to work with one database, you can easily work with any other one. All you have to do is change the provider-specific string (i.e., SQL, Oracle, Oledb, Odbc) on the Connection, Command, DataReader, and DataAdapter objects, depending on the data source you are working with.

Remember that the ADO.NET objects (Connection, Command, DataReader, and DataAdapter) have different prefixes depending on the provider, as shown below.

1. Connection:

SQLConnection, OracleConnection, OleDbConnection, OdbcConnection, etc.

The first important component of ADO.NET Architecture is the Connection Object. The Connection Object is required to connect with your backend database which can be SQL Server, Oracle, MySQL, etc. To create a connection object, you need at least two things. The first one is where is your database located i.e. the Machine name or IP Address or someplace where your database is located. And the second thing is the security credentials i.e. whether it is a Windows authentication or SQL Authentication i.e. user name and password-based authentication. So, the first is to create the connection object and the connection object is required to connect the front-end application with the backend data source.

Note: The connections should be opened as late as possible and should be closed as early as possible, as the connection is one of the most expensive resources.

The SqlConnection class in ADO.NET is fundamental for establishing a connection to a SQL Server database. It provides methods and properties to manage the connection, execute commands, and handle transactions. Here's an overview of the SqlConnection class and its usage:

Import the Namespace:

To use the SqlConnection class, you need to include the System.Data.SqlClient namespace in your code:

```
using System.Data.SqlClient;
```

Create a Connection String:

Before creating an instance of SqlConnection, you need a connection string specifying the details of the SQL Server instance you want to connect to. The connection string includes information

such as the server name, database name, authentication method, and credentials.

```
string connectionString = “Server=MyServerAddress;Database=MyDataBase;User
Id=MyUsername;Password=MyPassword;”;
```

Instantiate and Open Connection:

Create an instance of SqlConnection using the connection string and then open the connection using the Open() method:

```
SqlConnection connection = new SqlConnection(connectionString);
connection.Open();
```

Execute Commands:
Once the connection is open, you can create and execute SQL commands using the SqlCommand class. For example, executing a SELECT query:

```
string sqlQuery = "SELECT FirstName, LastName FROM Employees";
SqlCommand command = new SqlCommand(sqlQuery, connection);
SqlDataReader reader = command.ExecuteReader();
while (reader.Read())
{
Console.WriteLine(reader["FirstName"] + " " + reader["LastName"]);
}
reader.Close();
```

Close the Connection:

After you’re done with the connection, close it to release resources:

```
connection.Close();
```

Using Statement (Recommended):

To ensure that the connection is properly closed, it's a good practice to use the using statement, which automatically disposes of the resources even if an exception occurs:

```
using (SqlConnection connection = new SqlConnection(connectionString))
{
connection.Open();
// Execute commands and perform operations
} // Connection is automatically closed and disposed here
```

Error Handling:
Wrap your code with appropriate error handling to catch exceptions that might occur during database interactions. Always close the connection in a finally block to ensure it's closed regardless of exceptions.

The SqlConnection class supports various events, connection pooling, and timeouts. Handling connections carefully to avoid leaks and ensure efficient resource usage is important. Additionally, remember that hardcoding sensitive information, like credentials in the connection string, is not secure. Consider using configuration files or other secure methods for storing connection strings.

2. Command:

SQLCommand, OracleCommand, OleDbCommand, OdbcCommand, etc.

The Second important component of ADO.NET Architecture is the Command Object. When we talk about databases like SQL Server, Oracle, MySQL, etc., we know one thing, these databases only understand SQL. The Command Object is the component where you go and write your SQL queries. Later you take the command object and execute it over the connection. That means using the command object, you can fetch data or send data to the database i.e. performing the Database CRUD Operations.

Note: From the Command Object onwards, you can go in two different ways. One is you can go with the DataSet way and the other is, you can go with the DataReader way. Which way you need to choose, basically will depend on the situation.

The SqlCommand class in ADO.NET is critical for executing SQL commands and stored procedures against a data source, such as a SQL Server database. It's part of the ADO.NET library and provides

methods and properties to define, execute, and retrieve results from database commands. Here's an overview of the SqlCommand class and its usage:

Import the Namespace:

To use the SqlCommand class, you need to include the System.Data.SqlClient namespace in your code:

using System.Data.SqlClient;

Create a SqlConnection:

Before using SqlCommand, you need an open SqlConnection object to execute commands. You can create and open a connection using the SqlConnection class described in the previous response.

Create and Execute Commands:

You can create an instance of SqlCommand and associate it with an open connection to execute various types of commands:

SELECT Query:

For executing SELECT queries and retrieving data:

```
string sqlQuery = ""select * from student"";
SqlCommand command = new SqlCommand(sqlQuery, connection);
SqlDataReader reader = command.ExecuteReader();
while (reader.Read())
{
Console.WriteLine(reader["Name"] + ", " + reader["Email"] + ", " + reader["Mobile"]);
}
reader.Close();
```

Non-Query Command (INSERT, UPDATE, DELETE):

For executing commands that don't return data (e.g., INSERT, UPDATE, DELETE):

```
string insertQuery = "Insert into Student values (105, 'Ramesh', 'Ramesh@dotnettutorial.net',
'1122334455')";
SqlCommand insertCommand = new SqlCommand(insertQuery, connection);
int rowsAffected = insertCommand.ExecuteNonQuery();
```

Stored Procedures:

For executing stored procedures:

```
SqlCommand spCommand = new SqlCommand("StoredProcedureName", connection);
spCommand.CommandType = CommandType.StoredProcedure;
SqlParameter parameter = new SqlParameter("@ParameterName", SqlDbType.VarChar);
parameter.Value = "ParameterValue";
spCommand.Parameters.Add(parameter);
// Execute the stored procedure
SqlDataReader spReader = spCommand.ExecuteReader();
```

Parameterized Queries:

It's highly recommended to use parameterized queries to prevent SQL injection. You can add parameters to your command to safely pass values:

```
string sqlQuery = "SELECT * FROM Employees WHERE Department = @Department";
SqlCommand paramCommand = new SqlCommand(sqlQuery, connection);
paramCommand.Parameters.AddWithValue("@Department", "IT");
SqlDataReader paramReader = paramCommand.ExecuteReader();
```

Dispose of Resources:

Always close and dispose of resources when you're done with them:

```
command.Dispose();
connection.Close();
connection.Dispose();
```

Error Handling:

Wrap your code with appropriate error handling to catch exceptions that might occur during database interactions.

Using Statement (Recommended):

To ensure that resources are properly disposed of, use the using statement:

```
using (SqlConnection connection = new SqlConnection(connectionString))
{
connection.Open();
using (SqlCommand command = new SqlCommand(sqlQuery, connection))
{
// Execute commands and retrieve data
} // Command is automatically disposed here
} // Connection is automatically closed and disposed here
```

The SqlCommand class provides methods and properties for a variety of use cases, allowing you to interact with an SQL Server database efficiently and securely.

Methods of SqlCommand Class in C#:

The SqlCommand class in C# provides the following methods.

1. **BeginExecuteNonQuery():** This method initiates the asynchronous execution of the Transact-SQL statement or stored procedure that is described by this **System.Data.SqlClient.SqlCommand.**
2. **Cancel():** This method tries to cancel the execution of a **System.Data.SqlClient.SqlCommand.**
3. **Clone():** This method creates a new System.Data.SqlClient.SqlCommand object is a copy of the current instance.
4. **CreateParameter():** This method creates a new instance of a **System.Data.SqlClient.SqlParameter object.**
5. **ExecuteReader():** This method Sends the **System.Data.SqlClient.SqlCommand.CommandText** to the **System.Data.SqlClient.SqlCommand.Connection** and builds a **System.Data.SqlClient.SqlDataReader.**
6. **ExecuteScalar():** This method Executes the query and returns the first column of the first row in the result set returned by the query. Additional columns or rows are ignored.
7. **ExecuteNonQuery():** This method executes a Transact-SQL statement against the connection and returns the number of rows affected.
8. **Prepare():** This method creates a prepared version of the command on an instance of SQL Server.

9. **ResetCommandTimeout():** This method resets the CommandTimeout property to its default value.

Note: ExecuteReader, **ExecuteNonQuery**, and **ExecuteScalar** are the methods that are commonly used. Let us see three examples to understand these methods.

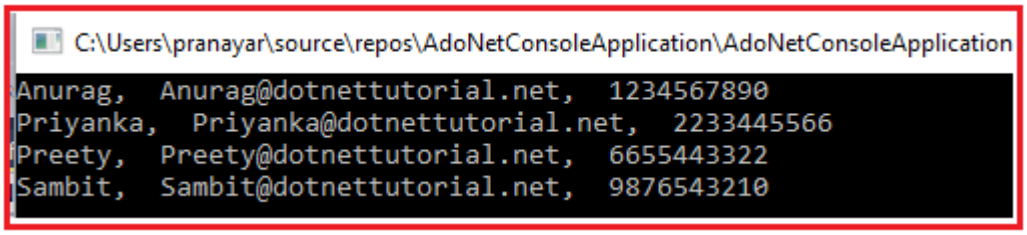
ExecuteReader Method of SqlCommand Object in C#:

As we already discussed, this method sends the CommandText to the Connection and builds a SqlDataReader. When your T-SQL statement returns more than a single value (for example, rows of data), then you need to use the ExecuteReader method. Let us understand this with an example. The following example uses the **ExecuteReader** method of the SqlCommand object to execute the T-SQL statement, which returns multiple rows of data.

```
using System;
using System.Data.SqlClient;
namespace AdoNetConsoleApplication
{
    class Program
    {
        static void Main(string[] args)
        {
            try
            {
            string ConString = "data source=.; database=StudentDB; integrated
            security=SSPI";
            using (SqlConnection connection = new SqlConnection(ConString))
            {
                // Creating SqlCommand objcet
                SqlCommand cm = new SqlCommand("select * from student",
            connection);
                // Opening Connection
                connection.Open();
                // Executing the SQL query
                SqlDataReader sdr = cm.ExecuteReader();
                while (sdr.Read())
                {
                    Console.WriteLine(sdr["Name"] + ", " + sdr["Email"] + ", " +
                sdr["Mobile"]);
                }
            }
            }
```

```
                    catch (Exception e)
                    {
                        Console.WriteLine("OOPs, something went wrong.\n" + e);
                    }
                    Console.ReadKey();
                }
            }
}
```

Once you execute the program, you will get the following output as expected.



Understanding the ADO.NET SqlCommand Object in C#:

In our example, we are creating an instance of the **SqlCommand** using the constructor, which takes two parameters, as shown in the image below. The first parameter is the **command text** we want to execute, and the second parameter is the connection object, which provides the database details on which the command will execute.

You can also create the **SqlCommand** object using the parameterless constructor, and later, you can specify the command text and connection using the **CommandText** and the **Connection** properties of the **SqlCommand** object, as shown in the example below.

```
using System;
using System.Data.SqlClient;
namespace AdoNetConsoleApplication
{
class Program
{
```

```
static void Main(string[] args)
{
try
{
string ConString = "data source=.; database=StudentDB; integrated security=SSPI";
using (SqlConnection connection = new SqlConnection(ConString))
{
// Creating SqlCommand objcet
SqlCommand cmd = new SqlCommand();
cmd.CommandText = "select * from student";
cmd.Connection = connection;
// Opening Connection
connection.Open();
// Executing the SQL query
SqlDataReader sdr = cmd.ExecuteReader();
while (sdr.Read())
{
Console.WriteLine(sdr["Name"] + ", " + sdr["Email"] + ", " + sdr["Mobile"]);
}
}
}
catch (Exception e)
{
Console.WriteLine("OOPs, something went wrong.\n" + e);
}
Console.ReadKey();
}
}
}
```

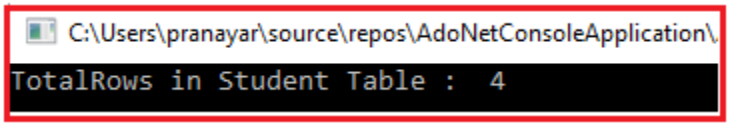
ExecuteScalar Method of SqlCommand Object in C#:

When your T-SQL query or stored procedure returns a single (i.e., scalar) value, then you need to use the **ExecuteScalar** method of the SqlCommand object in C#. Let us understand this with an example. Now, we need to fetch the total number of records present in the Student table. It will return a single value, so this is an ideal situation to use the **ExecuteScalar** method. The following example will retrieve the total number of records present in the **Student** table.

```
using System;
using System.Data.SqlClient;
namespace AdoNetConsoleApplication
{
class Program
{
```

```
static void Main(string[] args)
{
try
{
string ConString = "data source=.; database=StudentDB; integrated security=SSPI";
using (SqlConnection connection = new SqlConnection(ConString))
{
// Creating SqlCommand objcet
SqlCommand cmd = new SqlCommand("select count(id) from student", connection);
// Opening Connection
connection.Open();
// Executing the SQL query
// Since the return type of ExecuteScalar() is object, we are type casting to int datatype
int TotalRows = (int)cmd.ExecuteScalar();
Console.WriteLine("TotalRows in Student Table : " + TotalRows);
}
}
catch (Exception e)
{
Console.WriteLine("OOPs, something went wrong.\n" + e);
}
Console.ReadKey();
}
}
}
```

The return type of the ExecuteScalar method is an object, so here, we need to typecast it into an integer type. If you execute the above program, you will get the following output.



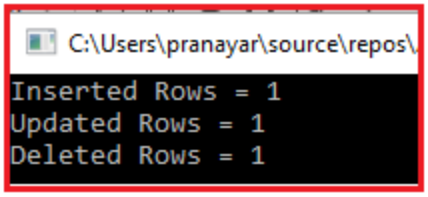
ExecuteNonQuery Method of ADO.NET SqlCommand Object in C#:

When you want to perform **Insert, Update, or Delete** operations and return the number of rows affected by your query, you need to use the **ExecuteNonQuery** method of the SqlCommand object in C#. Let us understand this with an example. The following example performs Insert, Update, and Delete operations using the **ExecuteNonQuery()** method.

```
using System;
using System.Data.SqlClient;
namespace AdoNetConsoleApplication
{
```

```
class Program
{
static void Main(string[] args)
{
try
{
string ConString = "data source=.; database=StudentDB; integrated security=SSPI";
using (SqlConnection connection = new SqlConnection(ConString))
{
SqlCommand cmd = new SqlCommand("insert into Student values (105, 'Ramesh',
'Ramesh@dotnettutorial.net', '1122334455')", connection);
connection.Open();
int rowsAffected = cmd.ExecuteNonQuery();
Console.WriteLine("Inserted Rows = " + rowsAffected);
//Set to CommandText to the update query. We are reusing the command object,
//instead of creating a new command object
cmd.CommandText = "update Student set Name = 'Ramesh Changed' where Id = 105";
rowsAffected = cmd.ExecuteNonQuery();
Console.WriteLine("Updated Rows = " + rowsAffected);
//Set to CommandText to the delete query. We are reusing the command object,
//instead of creating a new command object
cmd.CommandText = "Delete from Student where Id = 105";
rowsAffected = cmd.ExecuteNonQuery();
Console.WriteLine("Deleted Rows = " + rowsAffected);
}
}
catch (Exception e)
{
Console.WriteLine("OOPs, something went wrong.\n" + e);
}
Console.ReadKey();
}
}
}
```

Output:



So, in short, we can say that the SqlCommand Object in C# is used to prepare the command text (T-SQL statement or Stored Procedure) that you want to execute against the SQL Server database

and also provides some methods (ExecuteReader, ExecuteScalar, and ExecuteNonQuery) to execute those commands.

ADO.NET SqlCommand Class:

The SqlCommand class in ADO.NET is critical for executing SQL commands and stored procedures against a data source, such as a SQL Server database. It's part of the ADO.NET library and provides methods and properties to define, execute, and retrieve results from database commands. Here's an overview of the SqlCommand class and its usage:

Import the Namespace:

To use the SqlCommand class, you need to include the System.Data.SqlClient namespace in your code:

```
using System.Data.SqlClient;
```

Create a SqlConnection:

Before using SqlCommand, you need an open SqlConnection object to execute commands. You can create and open a connection using the SqlConnection class described in the previous response.

Create and Execute Commands:

You can create an instance of SqlCommand and associate it with an open connection to execute various types of commands:

SELECT Query:

For executing SELECT queries and retrieving data:

```
string sqlQuery = ""select * from student"";
SqlCommand command = new SqlCommand(sqlQuery, connection);
SqlDataReader reader = command.ExecuteReader();
while (reader.Read())
{
Console.WriteLine(reader["Name"] + ", " + reader["Email"] + ", " + reader["Mobile"]);
}
reader.Close();
```

Non-Query Command (INSERT, UPDATE, DELETE):

For executing commands that don't return data (e.g., INSERT, UPDATE, DELETE):

```
string insertQuery = "Insert into Student values (105, 'Ramesh', 'Ramesh@dotnettutorial.net', '1122334455')";
SqlCommand insertCommand = new SqlCommand(insertQuery, connection);
int rowsAffected = insertCommand.ExecuteNonQuery();
```

Stored Procedures:

For executing stored procedures:

```
SqlCommand spCommand = new SqlCommand("StoredProcedureName", connection);
spCommand.CommandType = CommandType.StoredProcedure;
SqlParameter parameter = new SqlParameter("@ParameterName", SqlDbType.VarChar);
parameter.Value = "ParameterValue";
spCommand.Parameters.Add(parameter);
// Execute the stored procedure
SqlDataReader spReader = spCommand.ExecuteReader();
```

Parameterized Queries:

It's highly recommended to use parameterized queries to prevent SQL injection. You can add parameters to your command to safely pass values:

```
string sqlQuery = "SELECT * FROM Employees WHERE Department = @Department";
SqlCommand paramCommand = new SqlCommand(sqlQuery, connection);
paramCommand.Parameters.AddWithValue("@Department", "IT");
SqlDataReader paramReader = paramCommand.ExecuteReader();
```

Dispose of Resources:

Always close and dispose of resources when you're done with them:

```
command.Dispose();
connection.Close();
connection.Dispose();
```

Error Handling:

Wrap your code with appropriate error handling to catch exceptions that might occur during database interactions.

Using Statement (Recommended):

To ensure that resources are properly disposed of, use the using statement:

```
using (SqlConnection connection = new SqlConnection(connectionString))
{
connection.Open();
using (SqlCommand command = new SqlCommand(sqlQuery, connection))
{
// Execute commands and retrieve data
} // Command is automatically disposed here
} // Connection is automatically closed and disposed here
```

The SqlCommand class provides methods and properties for a variety of use cases, allowing you to interact with an SQL Server database efficiently and securely.

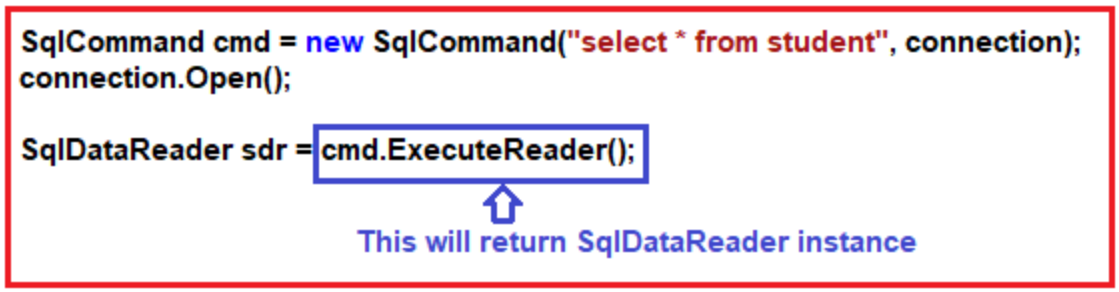
3. DataReader:

SQLDataReader, OracleDataReader, OleDbDataReader, OdbcDataReader, etc.

DataReader is a read-only connection-oriented recordset that helps us to read the records only in the forward mode. Here, you need to understand three things i.e. read-only, connection-oriented, and forward mode. Read-Only means using DataReader, we cannot Insert, Update, and Delete the data. Connection-Oriented means, it always requires an active and open connection to fetch the data. Forward mode means you can always read the next record, there is no way that you can read the previous record.

How do you create an instance of the ADO.NET SqlDataReader class in C#?

You can not create the instance of SqlDataReader using the new keyword. Then, the question is how we get or create the instance of the SqlDataReader class. To create the instance of the SqlDataReader class, what you need to do is call the **ExecuteReader** method of the **SqlCommand** object, which will return an instance of the **SqlDataReader** class, as shown in the image below.



Example to Understand ADO.NET SqlDataReader in C#:

We need to fetch all the data from the student table and need to display it in the console using SqlDataReader. The following code exactly does the same thing. In the below example, we use the **Read()** method of the **SqlDataReader** object to loop through the items of the **SqlDataReader** object. The Read method returns true as long as there are rows to read from the SqlDataReader object. If there are no more rows to read, this method will return false. In the example below, we are retrieving the data using the string key names, nothing but the column names returned by the select clause.

```
static void Main(string[] args)
{
try
{
string ConString = "data source=.; database=StudentDB; integrated security=SSPI";
using (SqlConnection connection = new SqlConnection(ConString))
{
// Creating the command object
SqlCommand cmd = new SqlCommand("select Name, Email, Mobile from student",
connection);
// Opening Connection
connection.Open();
// Executing the SQL query
SqlDataReader sdr = cmd.ExecuteReader();
//Looping through each record
while (sdr.Read())
{
Console.WriteLine(sdr["Name"] + ", " + sdr["Email"] + ", " + sdr["Mobile"]);
//Console.WriteLine(sdr[0] + ", " + sdr[1] + ", " + sdr[2]);
}
}
}
```

```
catch (Exception e)
{
Console.WriteLine("OOPs, something went wrong.\n" + e);
}
Console.ReadKey();
}
```

When to Use ADO.NET SqlDataReader Class in C#?

Here are some situations where you might consider using SqlDataReader:

- **High-Performance Data Reading:** Use SqlDataReader when performance is critical. It is optimized for fast, efficient data reading and faster than other methods like DataTable or DataSet because it does not load all data into memory at once.
- **Read-Only Data Access:** If you only need to read data from the database and don't need to perform any updates, inserts, or deletes, SqlDataReader is a suitable choice.
- **Large Volume Data Processing:** For processing large volumes of data, SqlDataReader is ideal because it reads data row-by-row, which means it uses less memory and can handle large datasets more efficiently.
- **Sequential Data Access:** When you need to access data sequentially, SqlDataReader is the way to go. It's a forward-only reader, meaning you can only read data in the order it is received from the database.
- **Data Binding in Web Applications:** In web applications, SqlDataReader can be used for simple data binding scenarios where the data is displayed in data-bound controls and no manipulation or updates are required.
- **Real-Time Data Display:** For scenarios where data needs to be fetched and displayed in real-time, like in dashboards or reporting tools, SqlDataReader provides an efficient way to retrieve and present data quickly.
- **Minimizing Memory Footprint:** If you want to minimize the memory footprint of your application, SqlDataReader is a good choice as it does not store the entire dataset in memory.
- **Streaming Data:** SqlDataReader is suitable for streaming data scenarios, where you process data as it's read without waiting for the entire set of data to be loaded.

4. DataAdapter:

SQLDataAdapter, OracleDataAdapter, OleDbDataAdapter, OdbcDataAdapter, etc.

The DataAdapter is one of the Components of ADO.NET which acts as a bridge between the command object and the dataset. What the DataAdapter does is, it takes the data from the command object and fills the data set.

ADO.NET Data Providers are libraries or components facilitating communication between your .NET application and specific data sources, such as databases. They are responsible for connecting to the data source, executing queries, and managing data retrieval and updates.

Each ADO.NET Data Provider is designed to work with a specific database or data source type. Some common ADO.NET Data Providers include:

- **SqlClient:** This is the ADO.NET Data Provider for Microsoft SQL Server. It provides optimized support for connecting to and interacting with SQL Server databases. The System.Data.SqlClient namespace contains classes like SqlConnection and SqlCommand.
- **OleDb:** The OleDb Data Provider allows you to connect to various data sources using the OLE DB technology. It can be used to connect to different databases, including Microsoft Access and other OLE DB-compatible data sources. The System.Data.OleDb namespace contains classes like OleDbConnection and OleDbCommand.
- **Odbc:** The Odbc Data Provider enables database connectivity using the ODBC (Open Database Connectivity) API. It supports a wide range of databases through ODBC drivers. The System.Data.Odbc namespace includes classes like OdbcConnection and OdbcCommand.
- **OracleClient:** This Data Provider connects to Oracle databases. Note that as of my last update in September 2021, Oracle has deprecated OracleClient, and using Oracle's own .NET driver or other alternatives is recommended.
- **SQLite:** SQLite is a self-contained, serverless, zero-configuration, transactional SQL database engine. There are ADO.NET Data Providers specifically designed for SQLite, allowing you to work with SQLite databases in your .NET applications.
- **MySqlClient:** This Data Provider is used to connect to MySQL databases. It allows .NET applications to interact with MySQL database servers.
- **Npgsql:** Npgsql is a popular Data Provider for PostgreSQL databases. It supports connecting to and working with PostgreSQL databases in .NET applications.
- **Other Third-Party Providers:** Third-party ADO.NET Data Providers are also available for various other databases and data sources. The database vendors or other developers often create these providers to enhance connectivity.

Example to Understand ADO.NET SqlDataAdapter in C#

Now, we need to develop an application where we will fetch all the data from the student table, and then we need to display the student data in the console using SqlDataAdapter in C# using ADO.NET. Let us first write the code, and then we will understand the code.

```
using System;
using System.Data;
using System.Data.SqlClient;
namespace AdoNetConsoleApplication
{
class Program
{
static void Main(string[] args)
{
try
{
string ConString = @"data source=LAPTOP-ICA2LCQL\SQLEXPRESS; database=StudentDB;
integrated security=SSPI";
using (SqlConnection connection = new SqlConnection(ConString))
{
SqlDataAdapter da = new SqlDataAdapter("select * from student", connection);
//Using Data Table
DataTable dt = new DataTable();
da.Fill(dt);
//The following things are done by the Fill method
//1. Open the connection
//2. Execute Command
//3. Retrieve the Result
//4. Fill/Store the Retrieve Result in the Data table
//5. Close the connection
Console.WriteLine("Using Data Table");
//Active and Open connection is not required
//dt.Rows: Gets the collection of rows that belong to this table
//DataRow: Represents a row of data in a DataTable.
foreach (DataRow row in dt.Rows)
{
//Accessing using string Key Name
Console.WriteLine(row["Name"] + ", " + row["Email"] + ", " + row["Mobile"]);
//Accessing using integer index position
//Console.WriteLine(row[0] + ", " + row[1] + ", " + row[2]);
}
```

```
Console.WriteLine("----------------");
//Using DataSet
DataSet ds = new DataSet();
da.Fill(ds, "student"); //Here, the datatable student will be stored in Index position 0
Console.WriteLine("Using Data Set");
//Tables: Gets the collection of tables contained in the System.Data.DataSet.
//Accessing the datatable from the dataset using the datatable name
foreach (DataRow row in ds.Tables["student"].Rows)
{
//Accessing the data using string Key Name
Console.WriteLine(row["Name"] + ", " + row["Email"] + ", " + row["Mobile"]);
//Accessing the data using integer index position
//Console.WriteLine(row[0] + ", " + row[1] + ", " + row[2]);
}
//Accessing the datatable from the dataset using the datatable index position
//foreach (DataRow row in ds.Tables[0].Rows)
//{
// Console.WriteLine(row["Name"] + ", " + row["Email"] + ", " + row["Mobile"]);
//}
}
}
catch (Exception e)
{
Console.WriteLine("OOPs, something went wrong.\n" + e);
}
Console.ReadKey();
}
}
}
```

Code Explanation:

Here, we create an instance of the **SqlDataAdapter** class using the constructor, which takes two parameters, i.e., the **SqlCommandText** and the **Connection** object. Then, we create an instance of DataSet and Datatable object. DataSet and DataTable are in-memory data stores that can store tables like a database. We will discuss **DataTable** and **DataSet** in our upcoming article.

Then, we call the **Fill()** method of the DataAdapter class. This method does most of the work behind us. It opens the connection to the database, executes the SQL command, fills the dataset and data tables with the data, and closes the connection. This method automatically handles the Opening and Closing of the database connections. The connection is kept open only as long as it is needed. That means once the Fill method completes its execution, then the connection closes

automatically. Finally, we are using DataRow to loop through each record and print the data on the console.
Once the dataset or data table is filled, then no active connection is required to read the data.

When to use ADO.NET SqlDataAdapter Class in C#?

SqlDataAdapter from ADO.NET is particularly useful in scenarios where you need to work with disconnected data models, perform batch updates, and synchronize changes between your application and a database. Here are some situations where you might consider using SqlDataAdapter:

- **Disconnected Data Operations:** If your application needs to work with data offline (i.e., without a continuous connection to the database), SqlDataAdapter is a good choice. It allows you to fill a DataSet or DataTable with data, manipulate it locally, and then apply changes back to the database.
- **Batch Updates:** SqlDataAdapter supports batch updates, where you can accumulate changes made to multiple rows in memory and then commit them to the database in a single batch. This can improve efficiency and reduce the number of round-trips to the database.
- **Data Binding:** If you're building data-bound user interfaces, SqlDataAdapter simplifies the process. You can bind controls directly to the DataTable within a DataSet, making it easier to display and manipulate data.
- **Caching and Offline Access:** You can use SqlDataAdapter to populate a DataSet with data, store it in memory, and allow users to work with the data even when offline or disconnected from the database.
- **Complex Data Manipulation:** When your application requires more than just simple data retrieval, and you need to perform inserts, updates and deletes on the data, SqlDataAdapter provides an efficient way to manage these changes.
- **Concurrency Control:** SqlDataAdapter can handle concurrency issues by using optimistic concurrency. It checks if the data being updated in the database matches the data in the DataSet before applying changes, preventing unintended overwrites.
- **Data Transformation:** You can use SqlDataAdapter along with other ADO.NET components to perform data transformation tasks, like changing data types, merging data from different sources, and more.
- **Performance:** In scenarios where you want to minimize the number of database round-trips while working with data, SqlDataAdapter allows you to fetch a bulk of data at once and work with it locally before applying changes back to the database.

Here's how SqlDataAdapter works:

- **Connection:** You establish a connection to your SQL Server database using SqlConnection.
- **Command:** You create one or more instances of SqlCommand that represent different SQL queries (e.g., SELECT, INSERT, UPDATE, DELETE) to be executed on the database.
- **DataAdapter Configuration:** You create and associate a SqlDataAdapter instance with the SqlCommand objects. This essentially tells the adapter which queries to use for different database operations.
- **Dataset:** You create an instance of DataSet or DataTable (which can hold multiple tables) in your application to store the retrieved data.
- **Data Retrieval:** You use the Fill() method of the SqlDataAdapter to execute the SELECT query and populate the DataSet or DataTable with the retrieved data.
- **Data Manipulation:** You can use the same SqlDataAdapter to update, insert, or delete data in the database. You modify the data in your DataSet or DataTable and then call the Update() method of the SqlDataAdapter to apply these changes to the database.

SqlDataAdapter is especially useful when working with disconnected data models, modifying the data locally, and then synchronizing those changes with the database. It simplifies the data retrieval and manipulation process, making it a preferred choice for scenarios involving more than just reading data.

DataSet:

The DataSet object in ADO.NET is not Provider-Specific. Once you connect to a database, execute the command and retrieve the data into the .NET application. The data can then be stored in a DataSet and work independently of the database. So, it is used to access data independently from any data source. The DataSet contains a collection of one or more DataTable objects.

It is a disconnected record set that can be browsed in both i.e. forward and backward mode. It is not read-only i.e. you can update the data present in the data set. Actually, DataSet is a collection of DataTables that holds the data and we can add, update, and delete data in a data table. DataSet gets filled by somebody called DataAdapter.

The DataSet is a collection of data tables. So, let's create a DataSet object and then add the two data tables (Customers and Orders) into the DataSet. Please have a look at the following image. Here, first, we created an instance of the DataSet and then added the two data tables using the Tables property of the DataSet object.

```
//Creating DataSet object
DataSet dataSet = new DataSet();

//Adding DataTables into DataSet
dataSet.Tables.Add(Customer);
dataSet.Tables.Add(Orders);
```

Fetch DataTable from DataSet:

Now, let us see how to fetch the data table from the dataset. You can fetch the data table from a dataset in two ways i.e. using the index position and using the table name (if provided).

Fetching DataTable from DataSet using index position:

As we first add the Customers table to DataSet, the Customer table Index position is 0. If you want to iterate through the Customer's table data, you could use a for-each loop to iterate, as shown in the image below.

```
//Fetching DataTable from dataset using the Index position
foreach (DataRow row in dataSet.Tables[0].Rows)
{
    Console.WriteLine(row["ID"] + ",  " + row["Name"] + ",  " + row["Mobile"]);
}
```

Fetching DataTable from DataSet using Name:

The second data table we added to the dataset is Orders, which will be added at index position 1. Further, if you notice, while creating the data table, we have provided a name for the data table, i.e., Orders. Now, if you want to fetch the data table from the dataset, then you can use the name instead of the index position. The following image shows how to fetch the data table using the name and looping through the data using a for each loop.

```
//Fetching DataTable from the DataSet using the table name
foreach (DataRow row in dataSet.Tables["Orders"].Rows)
{
    Console.WriteLine(row["ID"] + ",  " + row["CustomerId"] + ",  " + row["Amount"]);
}
```

When to use ADO.NET DataSet?

DataSet from ADO.NET is a versatile tool that's suitable for various scenarios where you need to work with complex data structures, manage relationships between multiple tables, and work with data in a disconnected environment. Here are some situations where you might consider using DataSet:

- **Complex Data Structures:** When your application involves multiple tables with relationships, DataSet provides a unified way to represent and manage these structures. It's particularly useful when working with data models that mirror a relational database.
- **Disconnected Data Manipulation:** DataSet is designed for scenarios where you need to work with data offline. You can fill a DataSet with data, manipulate it locally, and then apply changes back to the database using DataAdapter.
- **Data Binding with Relationships:** If you're building data-driven user interfaces that involve displaying data from multiple related tables, DataSet allows you to set up relationships between tables and bind controls to show the related data.
- **Data Consolidation:** If you're pulling data from multiple data sources or services and need to consolidate it for processing, DataSet can hold data from different sources and unify it for further manipulation.
- **Hierarchical Data:** If your data has a hierarchical structure, such as categories and subcategories, DataSet supports such relationships naturally, making it easier to work with parent-child data.
- **Data Caching and Offline Access:** In scenarios where network connectivity is intermittent or where data retrieval from the database is resource-intensive, DataSet can store a local copy of the data for offline usage.
- **Batch Updates:** When you need to perform batch updates that involve changes across multiple related tables, DataSet in conjunction with DataAdapter, enables you to apply changes to the database as a transaction.
- **Data Transformation:** If you need to transform or manipulate data before persisting it to the database or passing it to another layer of your application, DataSet provides tools to achieve these tasks.
- **Reporting and Data Analysis:** For tasks like generating reports or performing data analysis that require working with subsets of data, DataSet offers features like filtering, sorting, and grouping.
- **Data Serialization:** If you need to serialize your data for storage, exchange, or caching, DataSet can be serialized to formats like XML, which makes it easy to save and retrieve complex data structures.

Example to Understand ADO.NET DataSet using SQL Server Database in C#:

Our business requirement is to fetch all the data from the Customers table and then need to display it on the console. The following example exactly does the same using DataSet. In the below example, we created an instance of the DataSet and then fill the dataset using the Fill method of the data adapter object. The following example is self-explained, so please go through the comment lines.

```
using System;
using System.Data;
using System.Data.SqlClient;
namespace AdoNetConsoleApplication
{
class Program
{
static void Main(string[] args)
{
try
{
string ConnectionString = @"data source=LAPTOP-ICA2LCQL\SQLEXPRESS;
database=ShoppingCartDB; integrated security=SSPI";
using (SqlConnection connection = new SqlConnection(ConnectionString))
{
//Create the SqlDataAdapter instance by specifying the command text and connection object
SqlDataAdapter dataAdapter = new SqlDataAdapter("select * from customers", connection);
//Creating DataSet Object
DataSet dataSet = new DataSet();
//Filling the DataSet using the Fill Method of SqlDataAdapter object
//Here, we have not specified the data table name and the data table will be created at index
position 0
dataAdapter.Fill(dataSet);
//Iterating through the DataSet
//First fetch the Datatable from the dataset and then fetch the rows using the Rows property of
Datatable
foreach (DataRow row in dataSet.Tables[0].Rows)
{
//Accessing the Data using the string column name as key
Console.WriteLine(row["Id"] + ", " + row["Name"] + ", " + row["Mobile"]);
//Accessing the Data using the integer index position as key
//Console.WriteLine(row["Id"] + ", " + row["Name"] + ", " + row["Mobile"]);
}
```

```
}
}
catch (Exception ex)
{
Console.WriteLine($"Exception Occurred: {ex.Message}");
}
Console.ReadKey();
}
}
}
```

DataReader or DataSet in ADO.NET:

DataReader and DataSet are two distinct components in ADO.NET that serve different purposes for working with data. Each has its own strengths and weaknesses, and the choice between them depends on your specific application requirements. Let's compare DataReader and DataSet in ADO.NET:

DataReader:

- **Forward-Only Cursor:** DataReader provides a fast, forward-only, read-only cursor for fetching data from a database. It's optimized for retrieving data row by row.
- **Memory Efficiency:** DataReader is memory-efficient as it streams data from the database without loading the entire dataset into memory. This makes it suitable for large result sets.
- **Real-Time Access:** It's useful when you need to process data in real time, especially when memory consumption needs to be minimal.
- **Performance:** DataReader generally has better performance for read-heavy scenarios due to its lightweight nature and optimized data retrieval.
- **Read-Only Operations:** DataReader is designed primarily for reading data. You can't perform updates, insertions, or deletions using DataReader.
- **Disconnected Usage:** This is not suitable for disconnected scenarios where you want to work with data locally before updating the database.

DataSet:

1. **In-Memory Representation:** DataSet is an in-memory representation of a relational database, including multiple tables, relationships, and constraints.
2. **Disconnected Usage:** It's designed for disconnected scenarios where you can fill it with data from a database, manipulate it locally, and then apply changes back to the database using DataAdapter.

3. **Complex Data Structures:** Suitable for handling complex data models involving multiple tables, relationships, and constraints.
4. **Data Manipulation:** DataSet supports various data manipulation operations like sorting, filtering, grouping, and updating. It's versatile for both read and write operations.
5. **Data Binding:** DataSet supports data binding with controls, making it suitable for building data-driven user interfaces.
6. **Memory Consumption:** DataSet can consume more memory than DataReader because it loads data into memory. It might not be suitable for extremely large datasets.
7. **Performance:** DataSet might have slightly lower performance compared to DataReader due to the overhead of managing relationships and constraints.

Which one to use, DataReader or DataSet?

DataSet to use:

1. When you want to cache the data locally in your application so that you can manipulate the data.
2. When you want to work with disconnected architecture.

DataReader to use:

1. If you do not want to cache the data locally, then you need to use DataReader, which will improve the performance of your application.
2. DataReader works on connected-oriented architecture, i.e., requiring an open connection to the database.

DataTable:

The DataTable in C# is similar to the Tables in SQL. That means the DataTable will also represent the relational data in tabular form, i.e., rows and columns, and this data will be stored in memory. When we create an instance of DataTable, by default, it does not have a table schema, i.e., it does not have any columns or constraints by default. You can create the table schema by adding columns and constraints to the table. Only once you define the schema (i.e., columns and constraints) for the DataTable can you add rows to the data table. In order to use DataTable, you must have to include the **System.Data** namespace.

Note: The ADO.NET DataTable is a central object that can be used independently or can be used by other objects such as DataSet and the DataView.

How Do We Create a DataTable in C#?

In order to create a DataTable in C#, we first need to create an instance of the DataTable class. Then, we need to add DataColumn objects that define the data type to be held and insert DataRow objects that contain the data.

Let us discuss this step by step.

Step 1: Creating DataTable instance

Please have a look at the following image. Here, we use the constructor, which takes the table name as a parameter.

```
DataTable dataTable = new DataTable("Student");
```

The above code will create an empty data table for which the TableName property is set to Student. Later, you can use this property to access this data table from a DataTableCollection. Once the DataTable is created, the next important step is to add the data columns and define the schema for the columns.

Step 2: Adding DataColumn and Defining Schema

A DataTable is a collection of DataColumn objects referenced by the Columns property of the data table. A DataTable object is useless until it has a schema. You can create the schema by adding DataColumn objects and setting the column constraints. As we already know from SQL's point of view, Constraints are basically used to maintain data integrity. Let us see how to Create DataColumn and set the schema. The following image shows a data column that is created using all the available properties.

```
//Add the DataColumn using all properties
DataColumn Id = new DataColumn("ID");
Id.DataType = typeof(int);
Id.Unique = true;
Id.AllowDBNull = false;
Id.Caption = "Student ID";
dataTable.Columns.Add(Id);
```

The following image shows adding a Data Column using a few properties.

```
//Add the DataColumn few properties
DataColumn Name = new DataColumn("Name");
Name.MaxLength = 50;
Name.AllowDBNull = false;
dataTable.Columns.Add(Name);
```

The following image shows how to create a data column with the default properties.

```
//Add the DataColumn using defaults
DataColumn Email = new DataColumn("Email");
dataTable.Columns.Add(Email);
```

Creating Primary Key Column in Datatable:

Like SQL, the primary key of a DataTable object also consists of a column or columns that make up a unique identity for each data row. The following image shows how to set the PrimaryKey property on the Id column of the Student DataTable object.

```
//Setting the Primary Key
dataTable.PrimaryKey = new DataColumn[] { Id };
```

Creating DataRow Objects in C#:

Once you have created the DataColumns for the DataTable object, you can populate it by adding DataRow objects. You need to use the DataRow object and its properties and methods to retrieve, insert, update, and delete the values in the DataTable. The DataRowCollection represents the actual DataRow objects in the DataTable, and it has an Add method that accepts a DataRow object. The Add method is also overloaded to accept an array of objects instead of a DataRow object. The following image shows how to create and add data to the Student DataTable object.

```
//Add New DataRow by creating the DataRow object
DataRow row1 = dataTable.NewRow();
row1["Id"] = 101;
row1["Name"] = "Anurag";
row1["Email"] = "Anurag@dotnettutorials.net";
dataTable.Rows.Add(row1);
```

You can also add a new DataRow by adding the values in the image below.

```
//Adding new DataRow by simply adding the values
dataTable.Rows.Add(102, "Mohanty", "Mohanty@dotnettutorials.net");
```

Iterating the DataTable in C#:

You can use a for each loop to loop through the rows and columns of a data table. The following image shows how to enumerate through the rows and columns of a data table.

```
foreach (DataRow row in dataTable.Rows)
{
   Console.WriteLine(row["Id"] + ",  " + row["Name"] + ",  " + row["Email"]);
}
```

When Should We Use ADO.NET DataTable in C#?

DataTable from ADO.NET is a versatile tool that serves as an in-memory representation of tabular data. It's useful in various scenarios where you need to work with data in a structured format. Here are some situations where you might consider using DataTable:

1. **Disconnected Data Manipulation:** When you need to work with data in a disconnected manner (i.e., without a continuous connection to the database), DataTable provides a way to load and manipulate data locally before synchronizing changes back to the database.
2. **Data Binding:** If you're building data-driven user interfaces, you can use DataTable as a data source for controls like grids, lists, and combo boxes. This allows you to display and interact with data in a user-friendly way.
3. **Complex Data Manipulation:** In scenarios where you need to perform data manipulation operations like sorting, filtering, and grouping, DataTable provides built-in methods to achieve these tasks.
4. **Interchange Format:** DataTable can serve as a container for data during data exchange between different layers of an application or even between different applications. You can serialize it to formats like XML or JSON for this purpose.
5. **Local Caching:** If your application frequently accesses the same data, you can use DataTable to store a local copy of the data, reducing the need for frequent database queries.

6. **Small to Moderate Data Sizes:** DataTable is suitable for managing data of moderate size. It's not optimized for very large datasets, where other techniques like streaming and pagination might be more appropriate.
7. **Data Transformation:** You can use DataTable to transform data from one format to another before sending it to another layer of your application or storing it in a different format.
8. **Batch Updates:** If your application requires updating multiple records in a single transaction, you can accumulate changes in a DataTable and then perform a batch update to the database.
9. **Offline Access:** In cases where users might need to work with data while disconnected from the network (e.g., in mobile applications), DataTable can store the required data for local usage.
10. **Data Validation:** You can apply constraints and validation rules to columns within a Data Table to ensure data integrity before persisting changes to the database.

DataView Class:

The DataView class enables us to create different views of the data stored in a DataTable. This is most often used in data-binding applications. Using a DataView, we can expose the data in a table with different sort orders, and you can filter the data by row state or based on a filter expression.

Creating ADO.NET Dataview Instance in C#:

We can create a DataView Instance in C# in two different ways. They are as follows:

Using DataView Constructor: The constructor of the DataView class initializes a new instance of the DataView class and accepts the DataTable as an argument.

Syntax: DataView dataView = new DataView(dataTable);

Using DefaultView Property of the DataTable: We can also create a DataView by using the DefaultView property of the DataTable.

Syntax: DataView dataView = dataTable.DefaultView;

Displaying a Dataview in C#:

Using a for loop and using the integer index position, we can access the elements of a data view as follows:

Using For Loop to access the DataView

```
for(int i = 0; i<dataView.Count;i++)
{
    Console.WriteLine(dataView[i][“DataTableColumnName”]);
}
```

Using Foreach Loop to access the DataView

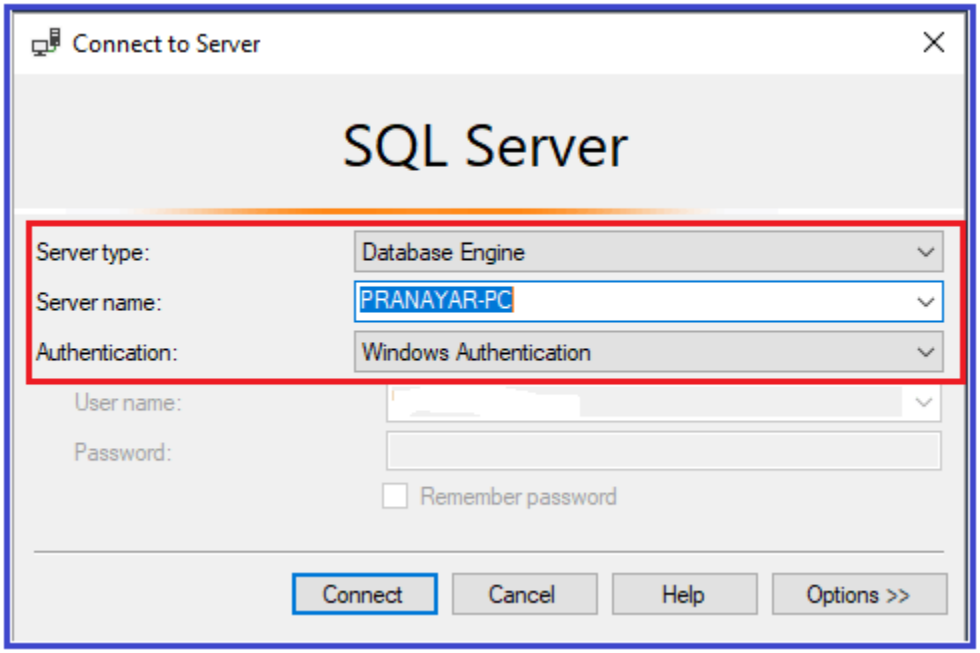
```
foreach (DataRowView rowView in dataView1)
{
    DataRow = rowView.Row;
    Console.WriteLine(row[“DataTableColumnName”]);
}
```

ADO.NET using SQL Server

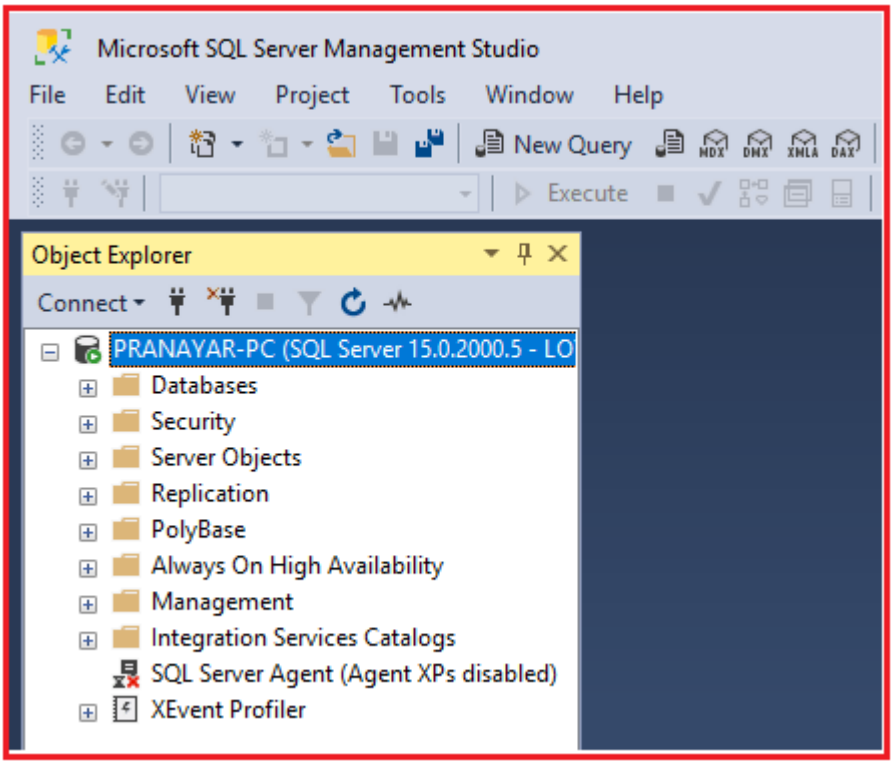
We are using the SQL Server Management Studio (SSMS) Tool to interact with the SQL Server.

Open SQL Server Management Studio Tool

Once you open SSMS (SQL Server Management Studio), It will prompt you to connect to the server window. Here, you need to provide the server name and authentication details (I am going with the Windows Authentication), select Database Engine as the server type, and finally, click the Connect button, as shown in the image below.

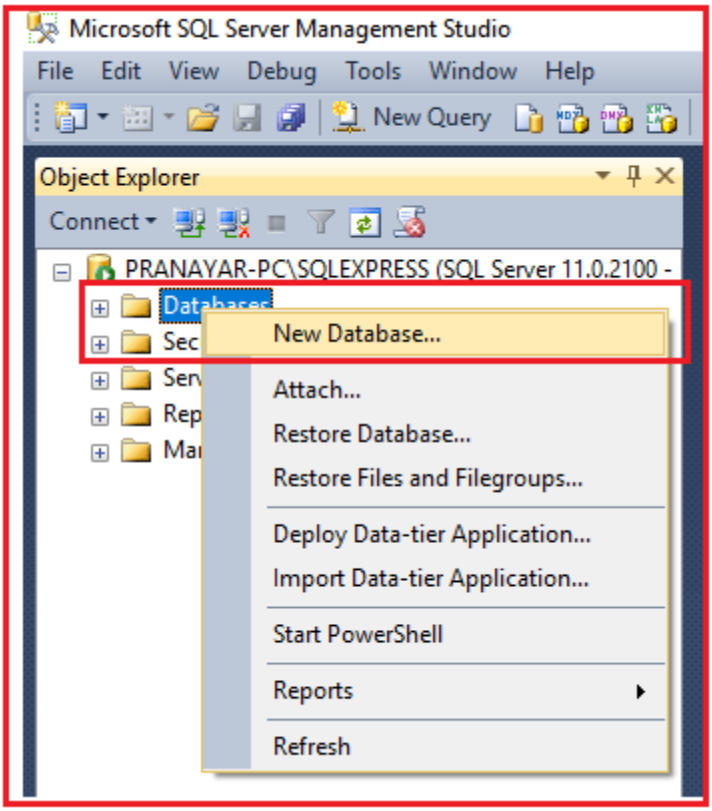


Once you click on the Connect button, it will connect to the SQL Server Database, and after a successful connection, it will display the following window.

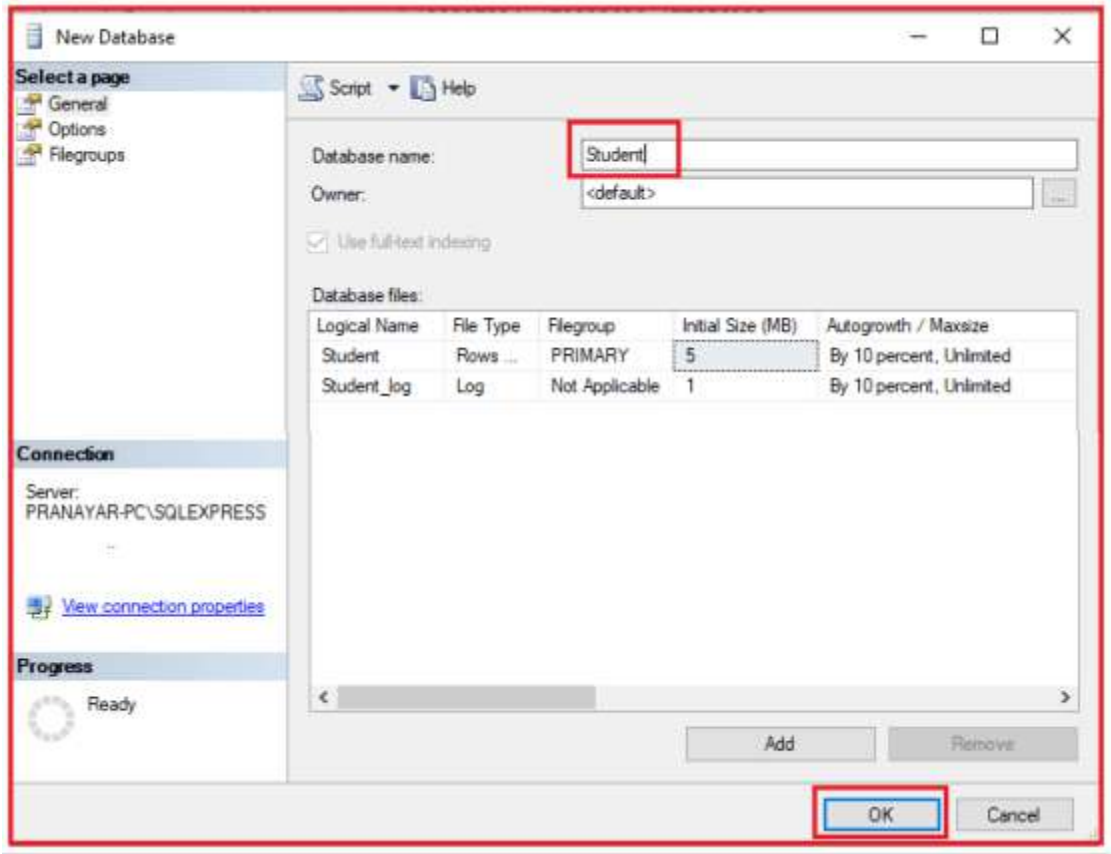


Creating Database in SQL Server

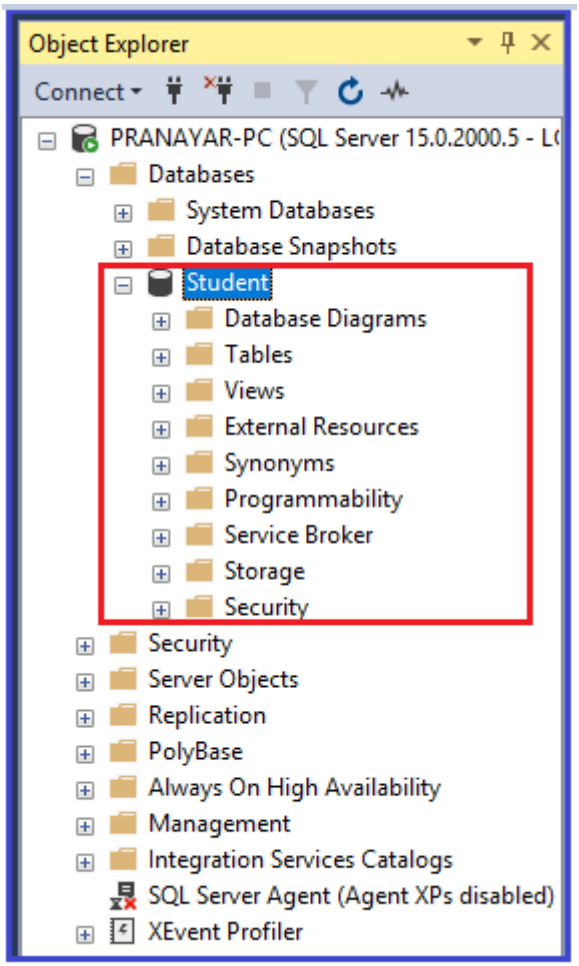
To create a database using GUI, you need to select the database option from Object Explorer and then right-click on it. It pops up an options menu, and here, you need to click on the New Database option, as shown in the below image.



Once you click on the **New Database** option, it will open the following New Database window. Here, you must provide the database name and click the **OK** button. Here, I created a database with the name **Student**. But it is up to you; you can provide any meaningful name you choose.



Once you click on the **OK** button, it will create a Student database, and you can see the Student database in the object explorer, as shown in the image below.



That's it. Our database part is over. Now, let us move to the ADO.NET part.

Establish a Connection to the SQL Server database and create a table using ADO.NET.

Once the Student Database is ready, let's move and create a table (Student table) using the ADO.NET Provider and C# Code. Open Visual Studio 2017 (you can use any version of Visual Studio), then create a new .NET console application project. Once you create the project, then modify the **Program.cs** class file as shown below.

using *System;*

using *System.Data.SqlClient;*

namespace *AdoNetConsoleApplication*

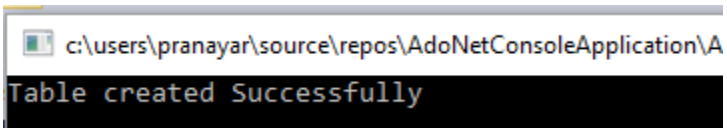
{

class Program

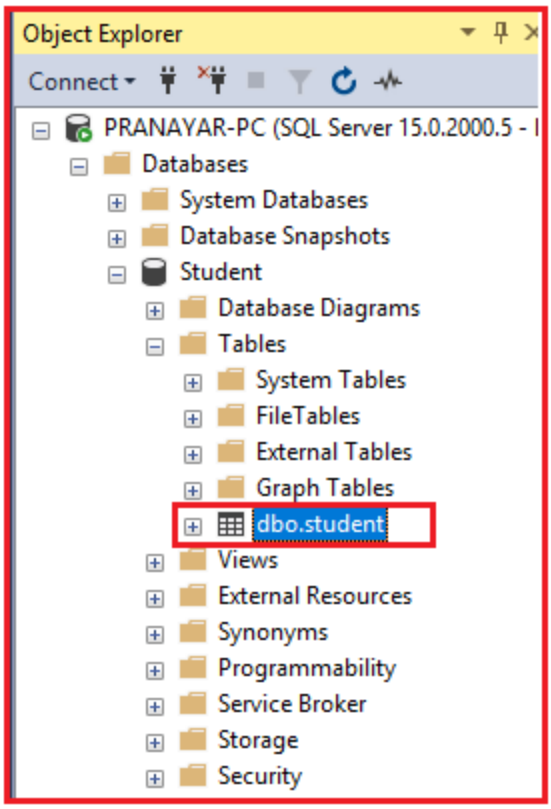
```
{
static void Main(string[] args)
{
new Program().CreateTable();
Console.ReadKey();
}
public void CreateTable()
{
SqlConnection con = null;
try
{
// Creating Connection
con = new SqlConnection("data source=.; database=student; integrated security=SSPI");
// writing sql query
SqlCommand cm = new SqlCommand("create table student(id int not null, name varchar(100),
email varchar(50), join_date date)", con);
// Opening Connection
con.Open();
// Executing the SQL query
cm.ExecuteNonQuery();
// Displaying a message
Console.WriteLine("Table created Successfully");
}
catch (Exception e)
{
Console.WriteLine("OOPs, something went wrong." + e);
```

```
}
// Closing the connection
finally
{
con.Close();
}
}
}
}
```

Now, execute the program, and you should see the following message on the console.



We can see the created table in Microsoft SQL Server Management Studio. It shows the created table as shown below.



See, we have the Student table within the Student database. As of now, the Student table is empty. Insert one record into the Student table using ADO.NET and C#.

Inserting Record using C# and ADO.NET:

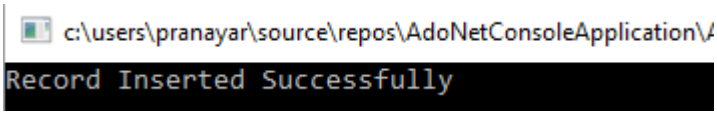
Please modify the **Program.cs** class file as shown below. Here, we will insert a record into the student table.

```
using System;

using System.Data.SqlClient;

namespace AdoNetConsoleApplication

{

class Program

{

static void Main(string[] args)

{
```

```
new Program().InsertRecord();
Console.ReadKey();
}
public void InsertRecord()
{
SqlConnection con = null;
try
{
// Creating Connection
con = new SqlConnection("data source=.; database=student; integrated security=SSPI");
// writing sql query
SqlCommand cm = new SqlCommand("insert into student (id, name, email, join_date) values
('101', 'Ronald Trump', 'ronald@example.com', '1/12/2017')", con);
// Opening Connection
con.Open();
// Executing the SQL query
cm.ExecuteNonQuery();
// Displaying a message
Console.WriteLine("Record Inserted Successfully");
}
catch (Exception e)
{
Console.WriteLine("OOPs, something went wrong." + e);
}
// Closing the connection
finally
```

```
{
con.Close();
}
}
}
}
```

Once you run the application, you will get the following output.



Retrieve Record using C# and ADO.NET.

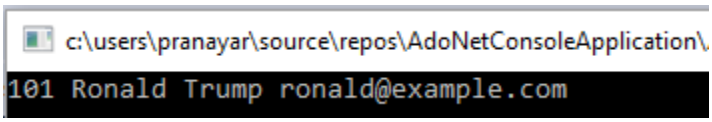
Here, we will retrieve the inserted data from the Student table of the student database. Please modify the **Program.cs** class file as shown below.

```
using System;
using System.Data.SqlClient;
namespace AdoNetConsoleApplication
{
class Program
{
static void Main(string[] args)
{
new Program().DisplayData();
Console.ReadKey();
}
public void DisplayData()
```

```
{
SqlConnection con = null;
try
{
// Creating Connection
con = new SqlConnection("data source=.; database=student; integrated security=SSPI");
// writing sql query
SqlCommand cm = new SqlCommand("Select * from student", con);
// Opening Connection
con.Open();
// Executing the SQL query
SqlDataReader sdr = cm.ExecuteReader();
// Iterating Data
while (sdr.Read())
{
// Displaying Record
Console.WriteLine(sdr["id"] + " " + sdr["name"] + " " + sdr["email"]);
}
}
catch (Exception e)
{
Console.WriteLine("OOPs, something went wrong." + e);
}
// Closing the connection
finally
{
```

```
con.Close();
}
}
}
}
```

You will get the following output when you run the above program.



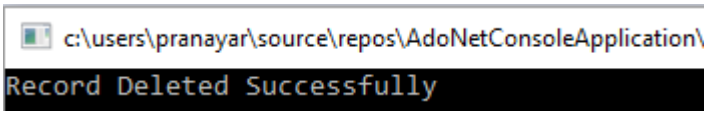
Deleting Record from SQL Server database using C# and ADO.NET

As of now, the student table contains one record. Let us delete that record using ADO.NET and C#. Please modify the Program.cs class file code as shown below which will delete the record from the Student table.

```
using System;
using System.Data.SqlClient;
namespace AdoNetConsoleApplication
{
class Program
{
static void Main(string[] args)
{
new Program().DeleteData();
Console.ReadKey();
}
public void DeleteData()
```

```
{
SqlConnection con = null;
try
{
// Creating Connection
con = new SqlConnection("data source=.; database=student; integrated security=SSPI");
// writing sql query
SqlCommand cm = new SqlCommand("delete from student where id = '101'", con);
// Opening Connection
con.Open();
// Executing the SQL query
cm.ExecuteNonQuery();
Console.WriteLine("Record Deleted Successfully");
}
catch (Exception e)
{
Console.WriteLine("OOPs, something went wrong." + e);
}
// Closing the connection
finally
{
con.Close();
}
}
}
}
```

It will display the following output once you execute the program.



If you verify the student table, you will see that the record is deleted. In this article, I didn't explain a single line of code intentionally. I will explain everything in detail in our next article.

Summary of ADO.NET using SQL Server:

Using ADO.NET with SQL Server involves establishing a connection to the SQL Server database, executing queries, retrieving and manipulating data, and managing transactions. Here's an overview of how you can use ADO.NET with SQL Server:

Import Required Namespaces:

using *System;*

using *System.Data;*

using *System.Data.SqlClient;*

Establish a Connection:

Create a connection string with the necessary details to connect to your SQL Server database, such as the server name, database name, and authentication credentials.

string connectionString = "Server=MyDatabaseServerAddress;Database=MyDataBase;User Id=MyUserName;Password=MyPassword;";

SqlConnection connection = new SqlConnection**(**connectionString**)**;

connection.Open**()**;

Execute Queries:

Use the SqlCommand class to create and execute SQL queries against the database. You can perform various types of queries, such as SELECT, INSERT, UPDATE, DELETE, and stored procedures.

```
string sqlQuery = "SELECT * FROM Employees";
SqlCommand command = new SqlCommand(sqlQuery, connection);
SqlDataReader reader = command.ExecuteReader();
while (reader.Read())
{
// Process the data from the reader
}
reader.Close();
```

Retrieve Data:

Use the SqlDataReader to retrieve and process data returned by SELECT queries. Iterate through the rows using the Read() method and access column values using indexer or GetString(), GetInt32(), etc., methods.

Update Data:

For data modification (UPDATE, INSERT, DELETE), you can use the ExecuteNonQuery() method of SqlCommand.

```
string updateQuery = "UPDATE Employees SET Salary = 50000 WHERE Department = 'IT'";
SqlCommand updateCommand = new SqlCommand(updateQuery, connection);
int rowsAffected = updateCommand.ExecuteNonQuery();
```

Use Transactions:

You can manage transactions using the SqlTransaction class. Transactions ensure that a group of operations is completed entirely or rolled back if an error occurs.

```
SqlTransaction transaction = connection.BeginTransaction();
try
{
// Perform data operations within the transaction
transaction.Commit();
}
catch (Exception ex)
{
transaction.Rollback();
}
```

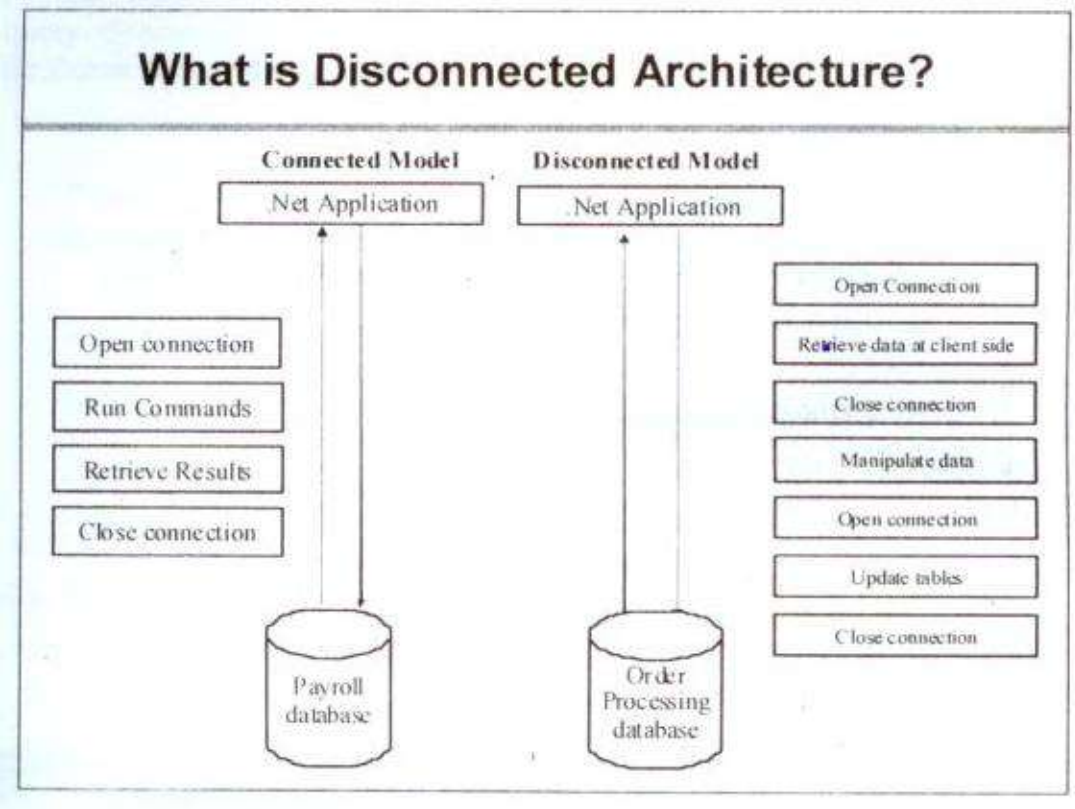
Close Connection:

Always close the connection when you're done using it to free up resources.

```
connection.Close();
```

Remember that ADO.NET involves more manual handling than higher-level ORMs like Entity Framework. While ADO.NET provides more control, it also requires writing more code. Depending on your project's needs and your familiarity with ADO.NET, you might use ADO.NET directly or explore alternatives like Entity Framework for more abstraction and ease of use.

Connected vs Disconnected Architecture in ADO.NET:



ADO.NET, a part of the .NET Framework, offers two distinct modes of interaction with a database: the connected and disconnected architectures. These architectures represent different ways of working with data sources.

Connected Architecture in ADO.NET:

In the connected architecture, an application interacts directly with the database using a connection. This means that the connection remains open as long as the application needs to interact with the database, and it's closed when the operations are completed. The connected architecture is primarily used for direct, real-time access to the database.

Components of Connected Architecture in ADP.NET:

- **Connection:** Establishes a link between the application and the database (e.g., SqlConnection for SQL Server). Ex.
- **Command:** Executes SQL commands or stored procedures (e.g., SqlCommand).
- **DataReader:** Provides a way to read a forward-only stream of data rows from a SQL Server database (SqlDataReader).

Features of Connected Architecture:

- **Direct Connection:** The application connects directly to the database for the duration of the data operation.
- **Real-Time Access:** Data is accessed and updated in real-time, making it suitable for scenarios where up-to-date data is critical.
- **Command Execution:** Executes SQL commands directly, such as SELECT, INSERT, UPDATE, and DELETE.

Use Cases of Connected Architecture:

- Real-time operations where immediate database interaction is required.
- Situations where data doesn't need to be persisted in the application for long-term manipulation.

Disconnected Architecture in ADO.NET

On the other hand, the disconnected architecture allows the application to interact with the database without maintaining a constant connection. Data is retrieved from the database and stored in an in-memory representation. Any changes made to this in-memory data can later be reconciled with the database.

Components of Disconnected Architecture in ADP.NET:

- **DataSet:** An in-memory Data set consisting of one or more DataTable objects.
- **DataAdapter:** Acts as a bridge between the DataSet and the data source for retrieving and saving data. It executes SQL commands and stores the results in the DataSet.
- **Connection:** Used for brief periods to fetch data or update the database.

Features of Disconnected Architecture

- **Disconnected Mode:** Connects to the database only to fetch and update data; the connection is closed otherwise.
- **In-Memory Data Representation:** Data is stored in memory in DataSet or DataTable objects.
- **Batch Updates:** Allows multiple changes to be made to the data in memory and then updated in the database in a single transaction.

Use Cases of Disconnected Architecture:

- Applications where data can be fetched, manipulated, and updated in the database later.
- Scenarios with intermittent or limited database connectivity.
- When bandwidth is a concern, data is fetched once and then worked on locally.

Comparison between Connected and Disconnected Architecture in ADO.NET

- **Performance:** The connected architecture is generally faster for immediate data operations but can be resource-intensive due to keeping the connection open. The disconnected architecture is more efficient regarding connection usage, as the connection is only open while reading or updating data.
- **Scalability:** Disconnected architecture is more scalable since it doesn't require a continuous connection to the database.
- **Concurrency:** Since data is manipulated offline in disconnected architecture, handling concurrency and potential conflicts is crucial when updating the database.
- **Complexity:** Connected architecture is often simpler to implement for straightforward database operations, while disconnected architecture requires additional steps to manage the local data cache and update the database.

DataGridView:

The `DataGridView` control in a C# Windows Forms application is a powerful and flexible tool for displaying and editing tabular data. You can bind it to various data sources, including databases and collections of custom objects, or you can populate it manually.

Basic setup in a Windows Forms application

1. **Create a new project:** Open Visual Studio, and create a new **Windows Forms App** project for C#.

2. **Add the `DataGridView` control:** From the **Toolbox**, drag and drop a `DataGridView` control onto your form.

3. **Name the control:** Select the `DataGridView` on your form and change its `(Name)` property in the Properties window (e.g., to `dataGridCustomers`).

Binding a `DataGridView` to a list of objects

This method is flexible, easy to manage, and ideal for creating a self-contained data grid without an external database.

1. Define a simple C# class

First, create a class to represent the data for each row. Use public properties so the `DataGridView` can automatically generate columns.

```
public class Customer
{
   public int CustomerId { get; set; }
   public string FirstName { get; set; }
   public string LastName { get; set; }
   public string City { get; set; }
}
Use code with caution.
```

2. Populate the grid

In your form's code-behind, create a `List<T>` of your custom objects and assign it to the `DataSource` property of your `DataGridView`.

```
private void Form1_Load(object sender, EventArgs e)
{
   // Create a list of customer objects
   List<Customer> customers = new List<Customer>();
   customers.Add(new Customer { CustomerId = 1, FirstName = "John", LastName = "Doe",
City = "New York" });
   customers.Add(new Customer { CustomerId = 2, FirstName = "Jane", LastName = "Smith",
City = "London" });
   customers.Add(new Customer { CustomerId = 3, FirstName = "Peter", LastName = "Jones",
City = "Paris" });

   // Set the data source
   dataGridCustomers.DataSource = customers;
}
```

Manually populating a `DataGridView`

This method provides full control over the columns and rows and does not require a binding source.

1. **Define the columns:**

```
// Set column count
dataGridCustomers.ColumnCount = 4;

// Set column headers
dataGridCustomers.Columns[0].Name = "ID";
dataGridCustomers.Columns[1].Name = "First Name";
dataGridCustomers.Columns[2].Name = "Last Name";
dataGridCustomers.Columns[3].Name = "City";
Use code with caution.
```

2. **Add rows manually:**

```
// Add a new row with data
string[] row1 = new string[] { "1", "John", "Doe", "New York" };
dataGridCustomers.Rows.Add(row1);

// Or add a new, empty row
dataGridCustomers.Rows.Add();
```

Adding and removing rows

Adding a new row

If the `DataGridView` is data-bound, you should add the new object to the underlying list. To enable the built-in "new row" at the bottom of the control, set the `AllowUserToAddRows` property to `true`.

Removing a selected row

1. **Remove from the grid:** To remove a row directly from the control, use `Rows.RemoveAt()`.
2. **Remove from the data source:** If you are using data binding, you must remove the item from the bound collection (e.g., a `List<T>` or `BindingList<T>`).

3. **Handle multiple selections:** When removing multiple selected rows, iterate backward to avoid index-shifting errors.

```
// Loop through all selected rows and remove them
foreach (DataGridViewRow row in dataGridCustomers.SelectedRows)
{
   if (!row.IsNewRow) // Ensure you don't delete the "new row" placeholder
   {
      dataGridCustomers.Rows.Remove(row);
   }
}
```

Handling user interaction

Cell click event

The `CellClick` event is useful for capturing when a user clicks a cell. This can be used to populate other controls with the selected row's data.

```
private void dataGridCustomers_CellClick(object sender, DataGridViewCellEventArgs e)
{
   if (e.RowIndex >= 0) // Check that a valid row is selected
   {
      // Get the selected row
      DataGridViewRow row = dataGridCustomers.Rows[e.RowIndex];

      // Access cell values
      string firstName = row.Cells["FirstName"].Value.ToString();
      // ... (populate textboxes with the data)
   }
}
```

Sorting

To enable or disable sorting, use the `SortMode` property for individual columns. By default, clicking a column header will automatically sort the data.

Customizing appearance and behavior

The `DataGridView` has a rich set of properties for customizing its look and feel.

- `DefaultCellStyle` : Change the font, colors, and other styles for cells.
- `AlternatingRowsDefaultCellStyle` : Create alternating row colors for better readability.
- `ReadOnly` : Make the entire grid read-only.
- `AllowUserToResizeColumns` / `AllowUserToResizeRows` : Prevent users from resizing columns or rows.
- `SelectionMode` : Change the selection behavior (e.g., `FullRowSelect` to select the entire row when clicking a cell).

To display data from a local SQL Server database in a DataGridView within a C# Windows Forms application, follow these steps:

1. Database Setup:

- Create a database and a table in SQL Server (e.g., using SQL Server Management Studio or Visual Studio's Server Explorer). Populate the table with some sample data.
- Note the server name (e.g., (localdb)\MSSQLLocalDB or your specific instance name) and the database name.

2. Create the Windows Forms Application:

- Open Visual Studio and create a new C# Windows Forms Application project.
- Drag a DataGridView control from the Toolbox onto your form. You can also add other controls like buttons for loading or refreshing data.

3. Establish the Database Connection and Load Data:

- Add using System.Data.SqlClient; to your form's code file.
- Create a method (e.g., LoadDataIntoDataGridView) to handle the data retrieval and binding.

- Inside this method, define a connection string to your local SQL Server database. This will include the Data Source (your server name), Initial Catalog (your database name), and Integrated Security=True for Windows authentication, or provide user credentials if using SQL Server authentication.
- Create a SqlConnection object using the connection string.
- Define a SQL SELECT query to retrieve data from your table.
- Create a SqlDataAdapter with the SELECT query and the SqlConnection.
- Create a DataTable object.
- Use the SqlDataAdapter to fill the DataTable with the retrieved data.
- Set the DataSource property of your DataGridView to the DataTable.

Example Code Snippet (within a Form Load event or button click):

Code

```
using System;
using System.Data;
using System.Data.SqlClient;
using System.Windows.Forms;

namespace DataGridViewApp
{
   public partial class Form1 : Form
   {
      public Form1()
      {
         InitializeComponent();
      }

      private void Form1_Load(object sender, EventArgs e)
      {
         LoadDataIntoDataGridView();
      }

      private void LoadDataIntoDataGridView()
      {
         string connectionString = "Data Source=(localdb)\\MSSQLLocalDB;Initial
```

```
Catalog=YourDatabaseName;Integrated Security=True"; // Replace YourDatabaseName
        string query = "SELECT * FROM YourTableName"; // Replace YourTableName

        using (SqlConnection connection = new SqlConnection(connectionString))
        {
            SqlDataAdapter adapter = new SqlDataAdapter(query, connection);
            DataTable dataTable = new DataTable();

            try
            {
                connection.Open();
                adapter.Fill(dataTable);
                dataGridView1.DataSource = dataTable;
            }
            catch (Exception ex)
            {
                MessageBox.Show("Error loading data: " + ex.Message);
            }
        }
    }
  }
}
```

4. Run the Application:

- Build and run your application. The DataGridView should now display the data from your specified SQL Server table.

UNIT 4: Section 2:

Crystal Reports

Crystal Reports is a business intelligence application used to design and generate reports from various data sources. It integrates well with Visual C# Windows applications, allowing for the creation of rich, data-driven reports.

Prerequisites

- **Visual Studio:** This tutorial assumes you have a recent version of Visual Studio installed (e.g., 2019 or 2022).
- **Crystal Reports Runtime:** You must install the SAP Crystal Reports runtime engine for your specific version of Visual Studio. Search the SAP website for the appropriate installer (e.g., "SAP Crystal Reports runtime engine for .NET Framework").
- **Data Source:** For this example, we will assume you have a SQL Server database.

Step 1: Install Crystal Reports via NuGet:

While the runtime engine is a separate installer, the libraries for development can be added through NuGet.

1. In Visual Studio, open your Windows Forms project.
2. Go to **Tools > NuGet Package Manager > Manage NuGet Packages for Solution...**
3. Click the **Browse** tab and search for `"Crystal Reports"`.
4. Find the package that corresponds to your Visual Studio version (e.g., `SAP Crystal Reports for Visual Studio 20xx`).
5. Select your project and click **Install**.

Step 2: Create the report file (.rpt**):**

1. In the **Solution Explorer**, right-click on your project, then select **Add > New Item**.
2. From the templates, choose **Reporting > Crystal Report**.
3. Name your report (e.g., `ProductReport.rpt`) and click **Add**.

4. In the **Crystal Reports Gallery**, select **As a Blank Report** and click **OK**.

Step 3: Connect to a data source:

For this example, we will connect to a local SQL Server database using a `DataSet`.

1. Open the newly created `ProductReport.rpt` file.
2. In the **Field Explorer** panel (typically on the right), right-click on **Database Fields** and select **Database Expert**.
3. In the `Database Expert`, expand **OLE DB (ADO)** and make a new connection.
4. Enter your server details and select your database. Choose **Integrated Security** or provide a username and password.
5. Expand the database and navigate to the table you want to use for the report (e.g., `tbl_products`). Move it to the **Selected Tables** pane.
6. Click **OK**. The table and its fields will now appear under **Database Fields** in the **Field Explorer**.

Step 4: Design the report layout:

Design the report by dragging and dropping fields from the **Field Explorer** onto the report sections in the designer window.

1. Expand **Database Fields** in the **Field Explorer**.
2. Drag and drop the fields you want to display (e.g., `ProductID`, `ProductName`, `UnitPrice`) onto the **Details** section.
3. Drag the same fields into the **Page Header** section to create column headers.
4. You can also add static text by dragging a **Text Object** from the **Field Explorer** to the report header for a report title.

Step 5: Format, group, and summarize data

Crystal Reports provides tools to make your data more organized and readable.

Grouping

1. Go to **Report > Group Expert**.
2. From the list of available fields, select the field you want to group by (e.g., `CategoryID`).
3. Choose the sorting order (e.g., `in ascending order`) and click **OK**.
4. New sections for the group header and footer will be added to your report. Drag the group field (`CategoryID`) to the **Group Header #1** section.

Summarizing

1. Right-click the field you want to summarize in the **Details** section (e.g., `UnitPrice`).
2. Select **Insert > Summary**.
3. In the `Insert Summary` dialog, choose a function like `Sum` and specify the summary location as the `Group Footer #1` or `Report Footer`.

Formatting

You can format fields by right-clicking them and selecting **Format Field**. This allows you to set font styles, colors, number formats, and more.

Step 6: Add a parameter to filter the report

Parameters allow users to filter data dynamically at runtime.

1. In the **Field Explorer**, right-click **Parameter Fields** and select **New**.
2. Name the parameter (e.g., `?CategoryID`) and set the **Type** to a number.
3. Click **OK**.
4. To link the parameter to the report's data, go to **Report > Select Expert**.

5. Choose the field to filter by (e.g., `tbl_products.CategoryID`), select the comparison operator `is equal to`, and then select your new parameter `?CategoryID`.

Step 7: Integrate the report into the C# application

1. Open the Windows Form where you want to display the report.
2. From the **Toolbox**, drag and drop a `CrystalReportViewer` control onto your form.
3. In the **Solution Explorer**, double-click on your form to open its code-behind file (`Form1.cs`).
4. Add the necessary namespaces at the top of your file:

```
using CrystalDecisions.CrystalReports.Engine;
using CrystalDecisions.Shared;
Use code with caution.
```

5. In the `Form_Load` event or a button click event, add the following code to load and display the report.

```
private void Form1_Load(object sender, EventArgs e)
{
   try
   {
      ReportDocument report = new ReportDocument();
      string reportPath = @"C:\YourProjectPath\ProductReport.rpt"; // Replace with your
report's full path

      report.Load(reportPath);

      // Pass the parameter value (e.g., from a TextBox)
      // ParameterDiscreteValue parameter = new ParameterDiscreteValue();
      // parameter.Value = 1; // Example: Pass category ID 1
      //
report.DataDefinition.ParameterFields["CategoryID"].CurrentValues.Add(parameter);
```

```
        // If a DataSet is used, populate and set it here
        // DataSet ds = GetDataFromDatabase();
        // report.SetDataSource(ds.Tables[0]);

        crystalReportViewer1.ReportSource = report;
        crystalReportViewer1.Refresh();
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message);
    }
}
```

Step 8: Export reports in C#

You can programmatically export reports to various formats like PDF, Excel, and more.

1. Add a button to your form (e.g., `btnExportPDF`).
2. Add an event handler for the button's `Click` event.
3. Use the `ExportToDisk()` method of the `ReportDocument` class.

```
private void btnExportPDF_Click(object sender, EventArgs e)
{
    try
    {
        ReportDocument report = new ReportDocument();
        string reportPath = @"C:\YourProjectPath\ProductReport.rpt"; // Replace with your report's full path
        report.Load(reportPath);

        // Set the export options
        string exportPath = @"C:\Exports\ProductReport.pdf";
        report.ExportToDisk(ExportFormatType.PortableDocFormat, exportPath);

        MessageBox.Show("Report exported successfully to PDF!");
```

```
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message);
    }
}
```

Step 9: Deploying the application

To deploy an application that uses Crystal Reports, the end-user's machine must have the correct Crystal Reports runtime installed.

Deployment steps:

1. Download the **SAP Crystal Reports runtime engine for .NET Framework** installer from the SAP website.
2. Include this installer with your application, so the user can run it before or during your application's installation.
3. You can also automate the installation of the runtime during your own application setup by creating a deployment project with a **Merge Module**. This will include the necessary components in your installer package.

Setup and Deployment of .net Applications:

Setup and deployment are critical for making a .NET application available to users by preparing the environment and transferring the application to a production server. This process ensures the application is reliable, scalable, and performs well on its target platform. Several methods exist for packaging and deploying a C# Windows application, each with different features for distribution, user experience, and automatic updates. The most common approaches involve using Visual Studio's Publish feature to create either a single-file executable, a ClickOnce installer, or an MSIX package.

Method 1: Publish a single-file executable

This is the simplest method and requires no special installer. All application files are bundled into a single executable file, which you can distribute directly to users.

Steps:

1. **Open your project** in Visual Studio.
2. In **Solution Explorer**, right-click your project and select **Publish**.
3. Choose a **publish target** of **Folder**, and then specify the location where the output files should be saved.
4. After the profile is created, click **Show all settings**.
5. In the Profile settings dialog, configure the following options:
    - **Deployment Mode:** Select `Self-contained`. This includes the .NET runtime with your application, so the user doesn't need to have it pre-installed.
    - **Target Runtime:** Select the appropriate platform, such as `win-x64` for 64-bit Windows.
    - **Produce single file:** Check this box to bundle all the application files into a single `.exe` file.
6. Click **Save**, then click **Publish**.
7. The single `.exe` file is created in the designated publish folder and is ready for distribution.

Method 2: Deploy using ClickOnce

ClickOnce is a deployment technology that can be used to create self-updating Windows applications. The user installs the app from a network file share or web server, and the app automatically checks for updates the next time it runs.

Steps:

1. **Open your project** in Visual Studio.
2. Right-click the project in **Solution Explorer** and select **Publish**.
3. Choose a **Folder** target and then select **ClickOnce**.
4. Specify the publish location, such as a local or network folder, and click **Next**.
5. In the Install location page, select where users will install the application. For example, if deploying from a website, enter the URL.
6. In the Settings page, you can configure automatic updates and other advanced options.
7. Click **Publish**.
8. Visual Studio generates a `setup.exe` bootstrapper and other application files in your publish location. Users can install the application by running the `setup.exe` file.

Method 3: Create an MSIX installer

MSIX is a modern Windows app package format that offers a secure and reliable packaging experience for all Windows applications. It provides a clean install and uninstall experience and can leverage features like automatic updates.

Steps:

1. **Install the extension:** In Visual Studio, navigate to **Extensions** > **Manage Extensions**, search for `Visual Studio Installer Projects`, and download it. Restart Visual Studio to complete the installation.

2. **Add a new project:** In Solution Explorer, right-click your solution, and select **Add** > **New Project**. Search for `Windows Application Packaging Project`, select it, and click **Next**.

3. **Add project references:** In the new packaging project, right-click the **Dependencies** folder and select **Add Project Reference**. Select your main C# Windows application and click **OK**.

4. **Set as startup project:** Right-click the new packaging project and select **Set as StartUp Project**. This allows you to debug the application within the MSIX container.

5. **Configure package details:** Double-click the `Package.appxmanifest` file in your packaging project to customize details like the display name and publisher.

6. **Create the package:** Right-click the packaging project and select **Publish** > **Create App Packages**. Choose **Sideloading** as the distribution method.

7. **Sign the package:** Select or create a signing certificate. A self-signed certificate can be used for testing, but a valid certificate is required for public distribution.

8. **Build and install:** Complete the wizard to generate the MSIX file. To install it, double-click the `.msix` file on the target machine.